

Smells na cadeia de fornecimento de software em projectos npm, aplicado a aplicações React

DEI.ISEP - Departamento de Engenharia Informática

2025/2026

1221267 Rafael Filipe Correia Santos

Smells na cadeia de fornecimento de software em projectos npm, aplicado a aplicações React

DEI.ISEP - Departamento de Engenharia Informática

2025/2026

1221267 Rafael Filipe Correia Santos



Licenciatura em Engenharia Informática

Junho de 2026

Orientador ISEP: **Isabel Azevedo (IFP)**

Declaração de Integridade

Declaro ter conduzido este trabalho académico com integridade.

Não plagiei ou apliquei qualquer forma de uso indevido de informações ou falsificação de resultados ao longo do processo que levou à sua elaboração.

Portanto, o trabalho apresentado neste documento é original e de minha autoria, não tendo sido utilizado anteriormente para nenhum outro fim.

Declaro ainda que tenho pleno conhecimento do Código de Conduta Ética do P.PORTO.

ISEP, 7 de junho de 2026

Resumo

O desenvolvimento de aplicações web, neste caso, aplicações React, dependem fortemente de bibliotecas externas. Este facto também pode causar problemas e perigos de segurança ligada à cadeia de fornecimento de software. Como dependências sem manutenção, dependências desnecessárias e que estejam a causar potencial perigo de segurança para toda a aplicação, que muitas vezes passam despercebidas. O objetivo passa por detetar *Software Supply Chain Smells* através da criação de uma ferramenta capaz de identificar dependências críticas, analisar o seu impacto e gerar relatórios de diagnóstico. Além disso, pretende-se avaliar o estado de saúde de aplicações React.

Esta ferramenta foi desenhada e analisada para detetar indicadores de problemas que poderão vir a surgir ou não (*smells*). Para isto foram incorporadas ferramentas para analisar os vários tipos de *smells*, como *smells* de segurança, depcheck, Dirty Waters, Dependency Sniffer e Snyk e uma ferramenta para analisar o *Software Bill of Materials*, Syft, que contém a lista de todas as dependências que se encontram num projeto React.

Pretende-se que esta ferramenta, através da agregação dos resultados dos *smells* de várias ferramentas que detetam diferentes indicadores de risco, venha a consolidar e enriquecer a segurança de uma aplicação React.

Palavras-chave: *Smells* na Cadeia de fornecimento de software, Análise de dependências, Cadeia de fornecimento de software, React, Dirty Waters, depcheck, Snyk, DependencySniffer, Syft, Software Bill of Materials

Abstract

The development of web applications, in this case React applications, heavily depends on external libraries. This fact can also cause security issues and risks related to the software supply chain. Such as unmaintained, unnecessary dependencies that may pose potential security risks to the entire application, often going unnoticed. The goal is to detect Software Supply Chain Smells by creating a tool capable of identifying critical dependencies, analyzing their impact, and generating diagnostic reports. Additionally, the aim is to assess the health of React applications.

This tool was designed and analyzed to detect indicators of problems that may or may not arise (*smells*). To achieve this, tools were incorporated to analyze the various types of *smells*, such as security *smells*, depcheck, Dirty Waters, Dependency Sniffer, and Snyk, as well as a tool to analyze the *Software Bill of Materials*, which contains the list of all dependencies present in a React project.

It is intended that this tool, through the aggregation of *smell* results from multiple tools that detect different risk indicators, will consolidate and enhance the security of a React application.

Agradecimentos

Agradeço á professora Isabel Azevedo (IFP) por toda a ajuda na orientação neste projeto de estágio, sendo incansável em ouvir qualquer dúvida e proposta que eu tinha e discutíamos a melhor solução. Pelas reuniões e propostas para seguir em frente no trabalho. Queria também agradecer a todos os professores e colegas que, durante a licenciatura cooperámos para adquirirmos o máximo de conhecimento possível.

Índice

1	Introdução	1
1.1	Enquadramento/Contexto	1
1.2	Descrição do Problema	1
1.2.1	Objetivos e Abordagem	2
1.2.2	Contributos	2
1.2.3	Planeamento do trabalho	3
1.3	Estrutura do relatório	4
2	Estado da Arte	5
2.1	OWASP CycloneDX	5
2.1.1	<i>JavaScript Object Notation (JSON)</i>	5
2.1.2	<i>Extensible Markup Language (XML)</i>	6
2.1.3	<i>Protocol Buffers (Protobuf)</i>	6
2.2	Ferramentas que suportam o Formato OWASP CycloneDX	7
2.2.1	Ferramentas para Geração	7
	Cyclone SBOM para npm	7
	Cyclone Generator (cdxgen)	8
	Syft	8
2.2.2	Ferramentas para Análise	9
	Dirty Waters	9
	Depcheck	10
	DependencySniffer	11
	Snyk	11
	Knip	12
	OpenSSF Scorecard	12
2.3	Dependências no Ecosistema NPM	12
2.4	<i>Software Supply Chain Smells</i>	14
2.4.1	Conceito	14
2.4.2	Tipos e Categorias	15
2.4.3	Relação a trabalhos já apresentados	16
3	Análise e Conceção da solução	17
3.1	Contexto	17
3.2	Domínio do problema	18
3.2.1	Modelo Domínio	18
3.2.2	Termos do domínio	20
3.2.3	Relações e Associações	20
3.2.4	Glossário	20
3.3	Requisitos	22
3.4	Conceção	24
3.4.1	Visão Geral da Arquitectura	26

Vista Lógica	26
Vista Física	29
Vista de Processos	29
Vista de Implementação	30
3.4.2 Iteração 1 — Solução Base	31
<i>Design</i>	31
Padrões de <i>Design</i> Aplicados	34
Aspectos Relevantes da Implementação	34
Avaliação Crítica	35
3.4.3 Iteração 2 — Arquitetura Extensível	35
Motivação e Alternativas Consideradas	36
<i>Design</i>	36
Padrões de Design Aplicados	39
Aspetos Relevantes da Implementação	40
3.4.4 Comparação entre as Iterações	40
4 Implementação da Solução	43
4.1 Descrição da implementação e configurações iniciais	43
4.1.1 Execução por ferramenta	45
Dirty Waters	45
Depcheck	45
Snyk	47
DependencySniffer	48
Syft	48
4.1.2 Processamento dos resultados das ferramentas	49
Dirty Waters	50
Depcheck	51
Snyk	52
DependencySniffer	52
Syft	53
4.1.3 Relatório dos <i>Smells</i> analisados	54
4.1.4 Aspetos Adicionais	56
4.2 Testes	57
4.2.1 Testes Unitários	58
4.2.2 Testes de Integração	60
4.2.3 Testes de Sistema	61
4.3 Avaliação da solução	62
4.3.1 Tempos de execução da ferramenta	63
4.3.2 <i>Smells</i> detetados pela ferramenta	65
4.3.3 Testes da configuração da ativação e desativação da deteção dos <i>smells</i>	66
5 Conclusões	69
5.1 Objetivos concretizados	69
5.2 Limitações e trabalho futuro	69
5.3 Apreciação final	70
6 Referências	71

Lista de Figuras

2.1	Exemplo Syft com informação legível para humanos	9
2.2	Gerar informações da ferramenta Dirty Waters.	10
2.3	Dados do relatório da ferramenta Dirty Waters do repositório TaskReact.	10
2.4	Dependências não utilizadas no projeto pela análise através da ferramenta dep-check	11
2.5	Resultado da ferramenta DependencySniffer	11
2.6	Resultado da ferramenta Snyk	12
3.1	Modelo Domínio	19
3.2	Diagrama de Casos de Uso	22
3.3	Vista Lógica Nível 1	27
3.4	Vista Lógica Nível 2	27
3.5	Vista Lógica Nível 3	28
3.6	Vista Física	29
3.7	Vista de Processos Nível 1 - Análise dos <i>smells</i>	30
3.8	Vista de Implementação Nível 2	31
3.9	Vista de Implementação Nível 3	31
3.10	Vista de Processos Nível 3 - Diagrama de sequência da iteração 1	32
3.11	Vista de Implementação Nível 3 - diagrama de classes da iteração 1	33
3.12	Vista de Processos Nível 3 - Diagrama de sequência da iteração 2	37
3.13	Vista de Implementação Nível 3 - diagrama de classes da iteração 2	38
4.1	Resultados com o SBOM em formato JSON	49
4.2	Exemplo de um sumário dos dados avaliados pela ferramenta Syft no relatório	54
4.3	Exemplos de relatórios gerados pela ferramenta	55
4.4	Exemplo de relatório gerado	56
4.5	Resultado da execução do comando de ajuda	57
4.6	Execução da ferramenta na linha de comandos	57
4.7	Execução dos testes	62
4.8	Relação entre o número de dependências e o tempo de execução	64
4.9	Reta de regressão linear entre o número de dependências e o tempo de execução	64
4.10	Output da ferramenta Depcheck	65
4.11	Output da ferramenta global	65
4.12	Output da ferramenta Snyk	65
4.13	Output da ferramenta global	66
4.14	Output da ferramenta Dirty Waters	66
4.15	Output da ferramenta global	66
4.16	Configurações "Code signature" ativada	67
4.17	Resultados com a configuração "Code signature" ativada	67
4.18	Configurações "Code signature" desativada	67
4.19	Resultados com a configuração "Code signature" desativada	67

Lista de Tabelas

1.1	Planeamento do trabalho	3
2.1	Smells detetados por Dirty Waters, Depcheck, DependencySniffer e Snyk	15
3.1	Glossário	21
3.2	Ferramentas existentes que detetam cada um dos <i>smells</i>	24
3.3	Ferramentas e <i>smells</i> explorados no projeto	25
3.4	Comparação da integração de ferramentas com e sem o padrão Adapter	35
3.5	Comparação entre as duas iterações	41
4.1	Tempo de execução para cada Repositório	63

Lista de Abreviações

AAA	A rrange A ct A ssert
CLI	C ommand L ine I nterface
CRA	C yber R esilience A ct
CVE	C ommon V ulnerabilities and E xposures
DEI	D epartamento E ngenharia I nformática
GRASP	G eneral R esponsibility A ssignment S oftware P atterns
IEC	I nternational E lectrotechnical C ommission
ISP	I nterface S egregation P rinciple
ISO	I nternational O rganization for S tandardization
JSON	J ava S cript O bject N otation
NPM	N ode P ackage M anager
NVD	N acional V ulnerability D atabase
OCF	O pen- C losed P rinciple
OWASP	O pen W eb A pplication S ecurity
SBOM	S oftware B ill of M aterials
XML	e Xtensible M arkup L anguage

Capítulo 1

Introdução

Este capítulo apresenta o projeto realizado durante o semestre no âmbito da unidade curricular Projeto - LEI-PROJ, do 3º ano da Licenciatura em Engenharia Informática, do Departamento de Engenharia Informática (DEI), do Instituto Superior de Engenharia do Porto (ISEP). O foco do projeto consiste na identificação e análise de *Software Supply Chain Smells* (Liu et al., 2025) em aplicações React.

1.1 Enquadramento/Contexto

Nos últimos anos, a segurança na cadeia de fornecimento de software (*Software Supply Chain*) tornou-se uma preocupação central na engenharia de software. Em setembro de 2019 a SolarWinds foi vítima de um ataque cibernético na cadeia de fornecimento de software que teve como consequência que cerca de 18 mil clientes instalaram indevidamente um conteúdo malicioso onde posteriormente, os atacantes utilizaram esse ponto para acessarem, roubarem dados sensíveis e realizar operações de espionagem contra organizações públicas e privadas a nível mundial. O ataque à SolarWinds evidencia fragilidades estruturais na cadeia de dependências de software (Fortinet, 2025).

Com o aumento de incidentes deste tipo a União Europeia aprovou um regulamento chamado Cyber Resilience Act (CRA), onde são estabelecidos requisitos obrigatórios de cibersegurança para produtos digitais (SoftwareSeni, 2026). Este regulamento motiva e reforça a necessidade de um controlo maior na gestão de dependências de terceiros.

O OWASP (OWASP Foundation, 2026) publica regularmente a lista OWASP Top 10, que identifica as 10 vulnerabilidades de segurança mais críticas em aplicações web. Esta lista é mantida e atualizada pela própria OWASP e serve como referência oficial para organizações na priorização de riscos e na adoção de práticas seguras no desenvolvimento e na gestão de aplicações web e respetivas cadeias de fornecimento.

Em aplicações modernas baseadas em componentes, como as desenvolvidas com React, a alta dependência de bibliotecas externas aumenta a superfície de ataque e a manutenção das aplicações.

1.2 Descrição do Problema

A segurança e manutenção de aplicações web baseadas em componentes podem ficar comprometidas por *Software Supply Chain Smells*, como apresentado na Secção 2.4 (Liu et al., 2025). Estes podem ser, por exemplo, dependências não mantidas, bibliotecas desatualizadas, vulnerabilidades críticas abertas e dependências muito pesadas. Estes indicadores

de problemas podem evidenciar riscos de segurança e custos de manutenção não esperada pelos programadores.

A verificação manual e a manutenção de dezenas de dependências num projeto React é complexa, ineficiente e propensa a erros. Atualmente, existe uma lacuna na disponibilização de um mecanismo leve e automatizado capaz de analisar ficheiros de manifesto, como por exemplo, o `package.json`, e identificar indicadores de problemas que poderão vir a ser problemas de segurança.

1.2.1 Objetivos e Abordagem

O presente projeto tem como principais objetivos o estudo do conceito de *Software Supply Chain Smells* (Liu et al., 2025), analisar indicadores e riscos associados a dependências em aplicações React, desenvolver um mecanismo automatizado para analisar ficheiros de manifesto como, por exemplo, `package.json`, identificar dependências potencialmente problemáticas, contribuir para a melhoria da segurança e manutenção de aplicações React. Esta ferramenta tem o objetivo de permitir a deteção automática destes problemas, reduzindo riscos de segurança e melhorando a manutenibilidade do software. Além disso, o projeto oferece benefícios práticos à organização, aumentando a confiabilidade das aplicações e sensibilizando os desenvolvedores para boas práticas de gestão de dependências, em alinhamento com recomendações como o OWASP Top 10 2025 (Noronha, 2025).

Para que os objetivos sejam atingidos será adotada a seguinte abordagem:

- Revisão e clarificação sobre o conceito de *Software Supply Chain Smells*;
- Análise de ferramentas existentes para deteção de dependências, indicadores de vulnerabilidades e gestão de dependências;
- Definição de critérios para identificar *smells* em aplicações React.
- Desenvolvimento de um protótipo capaz de analisar automaticamente ficheiros, por exemplo, `package.json`.
- Validação e verificação da ferramenta através da análise de vários projetos React, bem como a identificação de possíveis extensões e melhorias futuras, nomeadamente a aplicação da abordagem a outros ecossistemas e a deteção de novos *Software Supply Chain Smells*.

1.2.2 Contributos

O projeto apresenta contributos com relevância prática. Destaca-se a aplicação do conceito de *Software Supply Chain Smells* (Liu et al., 2025) no contexto de aplicações desenvolvidas em React, uma abordagem pouco explorada anteriormente.

O projeto disponibiliza um repositório, que permite a integração de várias ferramentas de análise de dependências, para assim ser gerado um relatório com os resultados dos *smells* definidos. Isto permite que o projeto seja extensível, com a adoção de outras ferramentas capazes de detetar outros *smells* e tornar a ferramenta cada vez mais completa e eficiente.

Além disso, contribui para a promoção de práticas de desenvolvimento mais seguras, diminuindo vulnerabilidades em aplicações web utilizadas por milhares de utilizadores e sensibiliza para riscos associados à cadeia de fornecimento de software, incentivando boas práticas de gestão e manutenção de dependências externas.

1.2.3 Planeamento do trabalho

Nesta subsecção é apresentado o planeamento definido para a execução do projeto com indicação das tarefas, datas e principais fases (*milestones*) relativas à ferramenta. As datas da realização de cada capítulo e também datas e marcos importantes durante todo o percurso do trabalho. A tabela 1.1 descreve todo o planeamento do trabalho:

Tabela 1.1: Planeamento do trabalho

Data Inicial	Data Final	O que é abordado	Capítulo relacionado
23/02/2026	09/03/2026	Análise dos conceitos, trabalhos e tecnologias relacionados com o <i>Software Supply Chain Smells</i> em projetos React.	Capítulo 1 e 2 (Introdução e Estado de Arte)
10/03/2026	23/03/2026	Preenchimento da Ata de Reunião; Experimentar ferramentas como, por exemplo, DirtyWaters para assim afeirir quais ferramentas serão usadas durante o desenvolvimento da solução.	Capítulo 2
24/03/2026	06/04/2026	Planeamento e designs da solução a desenvolver para análise de <i>Software Supply Chain Smells</i> em aplicações React.	Capítulo 3
07/04/2026	20/04/2026	Implementação da ferramenta automática para gerar e analisar o <i>Software Bill of Materials</i> (SBOM), para assim reportar padrões de risco e mostrar estatisticamente o estado da aplicação web; Avaliar o Ponto de Situação do trabalho realizado até então, avaliar se houve alterações à proposta inicial e quais as tarefas já realizadas e as que ainda estão por fazer.	Capítulo 4
21/04/2026	04/05/2026	Finalização da implementação da ferramenta final que analisa automaticamente as dependências de um projeto React e gera relatórios dos problemas identificados na cadeia de abastecimento. Começo da realização de testes à ferramenta.	Capítulo 4

Continua na próxima página

Data Inicial	Data Final	O que é abordado	Capítulo relacionado
05/05/2026	18/05/2026	Implementação e finalização dos testes unitários e testes de sistema, com testes a projetos reais de diferentes complexidades.	Capítulo 4
19/05/2026	01/06/2026	Ajustes finais e revisão do trabalho, preparação para a submissão do Projeto.	Capítulo 5
02/06/2026	08/06/2026	Preparação para a submissão do Projeto.	Todos
09/06/2026	03/07/2026	Apresentação do projeto realizado.	Todos

1.3 Estrutura do relatório

O relatório está dividido fundamentalmente em seis capítulos. Cada um dos capítulos tem um papel diferente pois cada um relata uma parte/etapa do processo tanto de aprendizagem como de desenvolvimento do trabalho e relatório.

No Capítulo 1, com o nome de "Introdução" são referidos aspetos importantes para a compreensão do tema, como, o contexto e descrição do problema, objetivos, abordagem, contributos e planeamento. Este permite ter uma ideia inicial do tema e contextualizar e ajudar a entender melhor o seu propósito.

O Capítulo 2 chamado "Estado de arte" tem como objetivo falar das tecnologias existentes e já conhecidas que têm relação com o trabalho, que, no caso, *Smells na cadeia de fornecimento de software em aplicações React*. Também é importante para que haja uma discussão de definições já revistas e trabalhos publicados por outros autores.

A análise e *design* da solução realizada está descrita no Capítulo 3. Seguindo as boas práticas, este capítulo descreve métodos, técnicas, algoritmos, tecnologias que são usadas na solução final, mas também o processo de análise e compreensão dos vários níveis de requisitos e funcionalidades da ferramenta.

No Capítulo 4 é descrito a implementação da solução relativa à análise de desenho presentes no capítulo anterior. Evidências da ferramenta realizada aparecem neste capítulo. Ainda neste capítulo são descritos testes para a validação da ferramenta e discussão sobre a mesma por completo.

As conclusões do trabalho realizado estão descritas no Capítulo 5 onde as conclusões baseiam-se nos resultados obtidos. Objetivos realizados, limitações da ferramenta realizada e uma opinião pessoal sobre o trabalho desenvolvido encontram-se nesta secção.

Capítulo 2

Estado da Arte

Neste capítulo é apresentado e explorado as principais ideias e noções relacionadas com o problema, mas também ferramentas suportadas em projetos React. São discutidas e analisadas ferramentas que suportam o formato OWASP CycloneDX para geração e análise do *Software Bill of Materials* (SBOM) para projetos React, e são discutidos os termos relevantes. Depois são consultados trabalhos relacionados com o tema como, por exemplo, o trabalho de Liu et al. com título “Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem”, para assim haver um melhor entendimento do problema. Por fim, ainda neste capítulo são clarificados termos e expressões de carácter relevante para o problema.

2.1 OWASP CycloneDX

OWASP CycloneDX (OWASP Foundation, 2026) é um padrão que tem como finalidade criar e partilhar *Software Bill of Materials* (SBOM), onde essencialmente cria uma lista detalhada de todos os componentes que integram um projeto. Isso inclui bibliotecas, frameworks, dependências, versões, licenças, entre outros. O padrão CycloneDX permite representar essa informação em formatos legíveis, como JSON, XML e Protocol Buffers. Tem como principais objetivos garantir a segurança pois permite procurar vulnerabilidades conhecidas nas dependências, está em concordância com as normas e boas práticas, como as recomendações da OWASP e pode ser integrado com ferramentas que verificam vulnerabilidades.

2.1.1 JavaScript Object Notation (JSON)

É um formato leve muito utilizado para troca de dados e tem uma estrutura fácil de entender para humanos e usa objetos através de pares **chave : valor**. Suporta vários tipos básicos de dados, como *string*, número, *array*, *null*, *boolean* e objetos (W3Schools, 2019). Tem a vantagem em comparação com outros formatos o facto de ser simples e legível, é suportado e usado por praticamente todas as linguagens de programação, mas em contrapartida, pode ser muito grande e dispendioso para, por exemplo, textos grandes, o que faz que o seu tamanho possa ser maior do que outros formatos binários (*Protocol Buffers*).

O extrato 2.1 mostra um exemplo do uso do formato *JSON* para representar os atributos de uma dependência, incluindo o *name*, *version*, *dep_type*, *source*, *purl* e os *smell_indicators*:

```
1 {  
2   "name": "request",  
3   "version": "2.88.2",  
4   "dep_type": "production",  
5   "source": "package.json",  
6   "purl": "pkg:npm/request@2.88.2",  
7   "smell_indicators": [  
8     "deprecated_dependency",  
9     "security_vulnerability"  
10  ]  
11 }
```

Excerto de Código 2.1: Exemplo de ficheiro JSON

2.1.2 Extensible Markup Language (XML)

XML é um formato de texto para dados que utiliza *tags*, `<tag>valor</tag>` e suporta atributos (W3Schools, 2019). É uma estrutura de dados que permite hierarquias complexas e é facilmente compreendido por humanos, pois são definidas pelo autor. Tem como vantagens ser muito flexível e extensível, possui sistema para validação de cada elemento e de compatibilidade mais antiga quando comparado com o *JSON*. Pode ser muito pesada a leitura para documentos mais extensos e a interpretação pelo programa é mais lenta.

O extrato 2.2 mostra um exemplo do uso do formato *XML* para representar os atributos de uma dependência, incluindo o *name*, *version*, *dep_type*, *source*, *purl* e os *smell_indicators*:

```
1 <dependency>  
2 <name>request</name>  
3 <version>2.88.2</version>  
4 <dep_type>production</dep_type>  
5 <source>package.json</source>  
6 <purl>pkg:npm/request@2.88.2</purl>  
7 <smell_indicators>  
8 <smell>deprecated_dependency</smell>  
9 <smell>security_vulnerability</smell>  
10 </smell_indicators>  
11 </dependency>
```

Excerto de Código 2.2: Exemplo de ficheiro XML

2.1.3 Protocol Buffers (Protobuf)

É um formato binário também conhecido como *Protobuf* desenvolvido pela Google para serialização de dados estruturados (Protocol Buffers Documentation, 2023). Para este protocolo é necessário definir a estrutura de dados pretendida no ficheiro ".proto", para gerar o código correspondente, na linguagem pretendida. Os campos têm tipo e número. Suporta diversas linguagens de programação: *Java*, *C++*, *PHP*, *Python*, entre outras. Tem como vantagens ser muito eficiente em tamanho e velocidade, que é menor que o *JSON* e *XML*, mas é de difícil compreensão humana devido ao facto de ser binário e requer compilador próprio para gerar o código em cada linguagem.

O extrato 2.3 mostra um exemplo do uso do formato *Protocol Buffers* para representar dados de uma dependência, incluindo o *name*, *version*, *depType*, *source*, *purl* e os *smellIndicators*:

```
1  syntax = "proto3";
2
3  message Dependency {
4      string name = 1;
5      string version = 2;
6      string depType = 3;
7      string source = 4;
8      string purl = 5;
9
10     repeated string smellIndicators = 6;
11 }
12
13 message DependencyRequest {
14     string name = 1;
15 }
16
17 message DependencyResponse {
18     Dependency dependency = 1;
19 }
20
21 service DependencyService {
22     rpc GetDependency (DependencyRequest) returns (DependencyResponse);
23     rpc CreateDependency (Dependency) returns (DependencyResponse);
24 }
```

Excerto de Código 2.3: Exemplo de ficheiro Protocol Buffers

O extrato de código mostra que no início é definida a versão do protocolo, neste caso *proto3*. Na *message* são definidos os dados trocados entre o cliente e o servidor, distinguindo-se entre o pedido (*DependencyRequest*) e a resposta (*DependencyResponse*).

Na *message Dependency* são representados os atributos associados a uma dependência, incluindo uma lista de *smell indicators*, utilizando a palavra-chave *repeated* para permitir múltiplos valores.

No *service* são definidos os métodos que podem ser invocados pelo cliente, neste caso *GetDependency* e *CreateDependency*.

2.2 Ferramentas que suportam o Formato OWASP CycloneDX

Como já foi referido anteriormente, o padrão OWASP CycloneDX é necessário para criar e partilhar SBOM. Existem ferramentas para geração e análise do *Software Bill of Materials* (SBOM) para projetos React.

Assim podemos separar as ferramentas em duas partes:

2.2.1 Ferramentas para Geração

Cyclone SBOM para npm

CycloneDX Node.js NPM Module (@cyclonedx/cyclonedx-npm) é uma ferramenta muito usada para projetos Node.js e JavaScript que utilizam o gestor de dependências NPM (Npm, 2026). Analisa o ficheiro *package.json* e as dependências instaladas. Usa o comando 'npm ls' para gerar a árvore de dependências e gera o SBOM no formato JSON ou XML. Inclui informações como o nome do componente, versão, dependências entre outros.

Comando para instalar a ferramenta CycloneDX:

```
npm install @cyclonedx/cyclonedx-npm
```

Neste exemplo um ficheiro sbom.json vai ser gerado com as informações do SBOM:

```
cyclonedx-npm --output-file sbom.json --output-format json
```

Cyclone Generator (cdxgen)

Ferramenta que suporta JavaScript, TypeScript e Node.js (JSR, 2026). Usada para gerar o SBOM e analisar código, dependências, containers e SaaS. Pode ser usada pela linha de comandos, biblioteca, servidor, entre outros. Exemplo da instalação pela linha de comandos para esta ferramenta:

```
npm install @cyclonedx/cdxgen
```

O SBOM pode ser gerado em JSON ou XML. Neste exemplo um ficheiro sbom.xml vai ser gerado com as informações do SBOM:

```
cdxgen -o sbom.xml -f xml
```

Syft

Outro exemplo de ferramenta é a Syft para gerar o *Software Bill of Materials* que recebe um container, um diretório onde estão esses mesmos arquivos e foi desenvolvido pela Anchore (Anchore, 2019). Compatível com o CycloneDX para projetos Node.js e containers.

Neste exemplo um ficheiro sbom.json vai ser gerado com as informações do SBOM:

```
syft scan dir:. -o cyclonedx-json > sbom.json
```

A figura 2.1 mostra um exemplo dos resultados da linha de comandos:

```

✓ Indexed file system
✓ Cataloged contents
├── ✓ Packages [8 packa
├── ✓ File metadata [0 locat
├── ✓ File digests [0 files
└── ✓ Executables [3 execu

[0000] WARN no explicit name and version provide
[Path: .]
[Path: \node_modules\@esbuild\win32-x64\esbuild]
Version:      UNKNOWN
Type:         binary
Found by:     pe-binary-package-cataloger

[frontendconverter]
Version:      0.0.0
Type:         npm
Found by:     javascript-lock-cataloger

[js-cookie]
Version:      3.0.5
Type:         npm
Found by:     javascript-lock-cataloger

```

Figura 2.1: Exemplo Syft com informação legível para humanos

2.2.2 Ferramentas para Análise

Após o ficheiro/documento SBOM ser gerado é necessária a sua análise. A análise do *Software Bill of Materials* tem como principal objetivo a procura *smells* e identificar riscos associados às dependências.

Foram analisadas algumas ferramentas para esta função, como a Dirty Waters (chains-project, 2025), depcheck (depcheck, 2024), DependencySniffer DataDog (2024), Snyk (Snyk,n.d.), Knip (webpro-nl, 2026) e OpenSSF Scorecard (OpenSSF, 2026) para colmatar algumas limitações da ferramenta Dirty Waters.

Dirty Waters

O recurso Dirty Waters é uma ferramenta que permite identificar riscos na cadeia de fornecimento de *software*. Esta baseia-se em vulnerabilidades conhecidas e como foram causadas, o nível de atualização e manutenção dos projetos, popularidade e atividade nos repositórios, o que está de acordo com alguns *smells* conhecidos e descritos nos trabalhos relatados, como dependências desatualizadas, com baixa manutenção, entre outros, aspetos abordados no estudo "*Dirty Waters - Detecting Software Supply Chain Smells e What are Weak Links in the npm Supply Chain*".

Seguindo o projeto no GitHub (<https://github.com/chains-project/dirty-waters.git>) foram seguidos vários passos definidos no README.md:

```
$Env:GITHUB_API_TOKEN = "token" # definir a GitHub API token
```

Instalação da ferramenta na versão 3.12 do Python pois a ferramenta ainda não suporta a versão 3.14:

```
py -3.12 -m pip install dirty-waters
```

A figura 2.2 mostra o comando para gerar os resultados da análise ao projeto React "TaskReact" e respetivo *output* na linha de comandos:

```
(dirty_env) PS C:\Users\Rafael\Desktop\TaskReactVite> dirty-waters -p RafiLS/TaskReact -v HEAD -pm npm --check-source-code
Table github_urls is up to date
Table pr_info is up to date
Table pr_reviews is up to date
Table tag_to_sha is up to date
Table package_analysis is up to date
Table commit_authors_from_tags is up to date
Table commit_authors_from_url is up to date
Table patch_authors_from_sha is up to date
Table user_commit is up to date
Table extracted_dependencies is up to date
Getting GitHub URLs: 100% | 648/648 [00:00<00:00, 2691.43it/s]
Analyzing packages: 100% | 648/648 [00:00<00:00, 2285.85it/s]
Report from static analysis generated at results\results_2026-03-25-17-55-23\00310d1c600e616e470c01a8f1aaba860b7065d9_static_summary.md
```

Figura 2.2: Gerar informações da ferramenta Dirty Waters.

Após isso vai ser gerado um ficheiro com informações detalhadas sobre a cadeia de fornecimento, como pacotes analisados, verificação de código fonte, estado das dependências, relação entre pacotes, *smells* ignorados e notas adicionais. A figura 2.3 mostra que foram obtidos resultados de 648 pacotes no qual, 0 sem URL de código fonte, 0 de repositório inconsistente, 648 pacotes não hospedados no GitHub, algumas dependências sem repositório de origem, a mesma dependência em múltiplos caminhos e nenhum *smell* ignorado.

Enabled Checks	
The following checks were requested project-wide:	
Check	Status
Source Code: <code>source_code</code>	✓
Source Code Sha: <code>source_code_sha</code>	✗
Deprecated: <code>deprecated</code>	✗
Forks: <code>forks</code>	✗
Provenance: <code>provenance</code>	✗
Code Signature: <code>code_signature</code>	✗
Aliased Packages: <code>aliased_packages</code>	✗
Ignore Configuration Summary	
Summary of Findings	
► How to read the results :book:	
Total packages in the supply chain: 648	
:heavy_exclamation_mark: Packages with no source code URL (▲▲▲▲): 0	
:no_entry: Packages with repo URL that is 404 (▲▲▲▲): 0	

Figura 2.3: Dados do relatório da ferramenta Dirty Waters do repositório TaskReact.

Depcheck

Como a ferramenta Dirty Waters tem limitações, por ela não detecta *bloated dependencies* nem várias versões do mesmo *package* foi usado o depcheck para o efeito. Esta ferramenta analisa dependências que não estão a ser utilizadas no projeto e/ou ferramentas que estão a ser usadas mas que não estão ligadas ao *package.json*, o que aumenta a complexidade do projeto e melhorar o desempenho, ou seja, diminuir potenciais riscos de segurança.

Comandos para uso da ferramenta depcheck para análise de dependências não utilizadas no projeto:

```
npm install --save-dev depcheck
```

Na figura 2.4 podem-se observar dependências não utilizadas pela análise desta ferramenta:

```
PS C:\Users\Rafael\Desktop\TaskReactVite> npx depcheck
Unused devDependencies
* @types/react
* @types/react-dom
* depcheck
* jest
* supertest
```

Figura 2.4: Dependências não utilizadas no projeto pela análise através da ferramenta depcheck

DependencySniffer

DependencySniffer é uma ferramenta que usa o depcheck como base utilizada para detetar e analisar *smells* relacionados com dependências em projetos JavaScript. Para além dos *smells* previamente referidos, esta ferramenta permite identificar potenciais riscos como *pinned dependencies*, *restrictive dependencies*, *permissive dependencies*, dependências baseadas em URL e a ausência de ficheiro package-lock. Adicionalmente, possibilita a identificação de riscos associados à gestão inadequada de dependências, contribuindo assim para uma maior robustez, manutenção e organização do projeto.

Comandos para instalar a ferramenta e executar a ferramenta DependencySniffer para o projeto TaskReactVite:

```
git clone https://github.com/abbasjavan/DependencySniffer.git
```

```
python DepSniffer.py TaskReactVite
```

A figura 2.5 mostra os resultados da ferramenta DependencySniffer para o projeto TaskReactVite:

	Dependency	Constraint	Smell
0	react	^16.10.1	none
1	jquery	3.4.1	none
2	lodash	~4.17.15	none
3	jest	^25.0.0	none
4	prettier	1.19.1	none
5	moment	>=2.24.0	none
6	mocha	*	none
7	mssql	git://github.com/patriksimek/node-mssql	none

Figura 2.5: Resultado da ferramenta DependencySniffer

Snyk

Com o objetivo de complementar a deteção de outros *smells*, como a execução de *install scripts*, a verificação de licenças e a análise de dependências transitivas, existe também a ferramenta Snyk. Esta ferramenta possibilita a análise de vulnerabilidades de segurança associadas às dependências do projeto, contribuindo para uma avaliação mais rigorosa da sua segurança.

Comandos para instalar a ferramenta e executar a ferramenta Snyk:

```
npm install -g snyk
```

```
snyk test
```

A figura 2.6 mostra os resultados da ferramenta Snyk para o projeto TaskReactVite:

```
Tested 34 dependencies for known issues, found 3 issues, 3 vulnerable paths.

Issues to fix by upgrading:

Upgrade axios@1.13.5 to axios@1.15.0 to fix
  X Unintended Proxy or Intermediary ('Confused Deputy') (new) [Medium Severity][https://security.snyk.io/vuln/
  SNYK-JS-AXIOS-15965856] in axios@1.13.5
    introduced by axios@1.13.5
  X HTTP Response Splitting (new) [High Severity][https://security.snyk.io/vuln/SNYK-JS-AXIOS-15969258] in axio
  s@1.13.5
    introduced by axios@1.13.5

Issues with no direct upgrade or patch:
  X Improper Removal of Sensitive Information Before Storage or Transfer [Medium Severity][https://security.sny
  k.io/vuln/SNYK-JS-FOLLOWREDIRECTS-16832162] in follow-redirects@1.15.11
    introduced by axios@1.13.5 > follow-redirects@1.15.11
    This issue was fixed in versions: 1.16.0

Organization:      rafils
Package manager:   npm
Target file:       package-lock.json
Project name:      taskreactvite
Open source:       no
Project path:      C:\Users\Rafael\Desktop\TaskReactVite
Licenses:          enabled
```

Figura 2.6: Resultado da ferramenta Snyk

Knip

A ferramenta Knip é uma ferramenta focada na detecção de código, dependências não utilizadas, dependências não declaradas e dependências transitivas em projetos JavaScript e TypeScript, incluindo aplicações React. Esta ferramenta é parecida com uma anteriormente mencionada, o Depcheck, sendo esta, mais simples de implementar, mais pesada e mais configurável.

OpenSSF Scorecard

O OpenSSF Scorecard é uma ferramenta que avalia práticas de segurança em projetos *open-source*. A ferramenta analisa diferentes métricas relacionadas com segurança e manutenção, como a utilização de integração contínua, assinaturas de *commits* e atividade do repositório. No contexto dos *smells*, o OpenSSF Scorecard permite identificar *smells* associados à qualidade e confiabilidade das dependências utilizadas num projeto React.

Assim, estas ferramentas têm como principal objetivo detetar *smells* relacionados com dependências e não detetar objetivamente esses mesmos problemas. Com base dos artigos estudados, os principais indicadores que podem afetar o sistema são, dependências desatualizadas, dependências não utilizadas, múltiplas versões da mesma biblioteca, com baixa manutenção, pacotes com baixa popularidade e a presença de vulnerabilidades conhecidas.

2.3 Dependências no Ecossistema NPM

As vulnerabilidades e os problemas dessa propagação são um ponto crítico abordado no trabalho de Liu et al. com título *Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem*, (Liu et al., 2022). Neste estudo, os autores analisam de que forma as vulnerabilidades se propagam ao longo das árvores de dependência no

contexto do Node Package Manager (NPM), muito utilizado no desenvolvimento de aplicações baseadas em Node.js.

Árvores de dependência são uma representação hierárquica de todos os pacotes utilizados, neste caso, no contexto NPM que inclui dependências diretas (pacotes instalados explicitamente) e dependências indiretas (pacotes necessários pelos pacotes instalados), gerando assim uma ramificação no projeto.

O trabalho demonstra que muitas das vulnerabilidades não são introduzidas diretamente pelos programadores, mas sim herdadas através de dependências transitivas, isto é, um pacote ou biblioteca que o projeto não inclui diretamente, mas que é necessário porque é usado por outra dependência do projeto. Ou seja, um projeto pode tornar-se mais vulnerável devido a bibliotecas de terceiros incluídas indiretamente, aumentando significativamente a superfície de ataque (Codacy, 2024).

São identificados padrões de propagação e a evolução temporal dessas vulnerabilidades, para assim, compreender o impacto nas dependências. Essa análise permitiu identificar padrões de propagação e a forma de como vulnerabilidades em pacotes muito utilizados se espalham rapidamente para múltiplos projetos dependentes. O estudo contém métricas para avaliar o impacto e profundidade das dependências, como a centralidade dos pacotes e frequência das atualizações.

Pacotes com elevada centralidade são componentes que ocupam uma posição central na rede de dependências no sistema NPM, ou seja, é um pacote onde muitas bibliotecas ou projetos dependem dele. Se este pacote conter vulnerabilidades, essa vulnerabilidade vai ser transmitida direta ou indiretamente para outros projetos. Liu et al. analisaram esses pacotes para entender quais os pontos críticos no ecossistema NPM.

Liu et al. observaram também que a correção de vulnerabilidades para além de ser demorada, nem sempre reduz o risco de forma imediata, uma vez que muitos projetos continuam a utilizar versões desatualizadas causadas por falta de monitorização e compatibilidade.

Sendo assim Liu et al. observaram:

- Pacotes NPM e projetos dependentes;
- Árvore de dependências;
- Vulnerabilidades conhecidas;
- Métricas para avaliar a centralidade dos componentes;
- Evolução temporal das vulnerabilidades;
- Impacto geral quanto ao número de projetos afetados por cada vulnerabilidade;

O trabalho avaliou vulnerabilidades conhecidas CVEs (*Common Vulnerabilities and Exposures*) na base de dados NVD (*National Vulnerability Database*). O estudo demonstrou que essas vulnerabilidades estão amplamente presentes nas dependências do NPM, afetando mais de 25% das versões em 20% dos pacotes. O estudo revelou que a maior parte das vulnerabilidades (93%) são colocadas nas árvores de dependência antes de ser descoberta e que cerca de 40% delas não conseguem ser totalmente eliminadas.

Investigaram também vulnerabilidades que se introduzem e se propagam ao longo do tempo nas árvores de dependências do NPM, e para isso construíram um grafo completo de todas as versões do NPM e simularam a propagação ao longo do tempo e identificaram que

demora cerca de um ano para que a maioria das vulnerabilidades seja removida, cerca de 40% permanecem ativas, mesmo após atualizações.

O impacto geral das vulnerabilidades quanto ao número de projetos afetados por cada falha de segurança, mostra que para além dos resultados mencionados anteriormente, cerca de 30% das versões afetadas estão em dependências diretas, não corrigidas. Demonstrando um efeito que pode escalar e tornando certos pacotes críticos para a segurança de todo o projeto/ecossistema.

Este estudo serviu para mostrar que muitas vulnerabilidades são herdadas através de dependências transitivas e que a sua correção é demorada e complicada, e que nem sempre reduz completamente o risco, mesmo com monitorização e atualização frequente.

2.4 Software Supply Chain Smells

2.4.1 Conceito

Software Supply Chain Smells, *Dependency Smells* ou *Weak Link Signals* são terminologias usadas por diferentes autores para descrever indícios de risco na cadeia de fornecimento de software, que servem como indicadores de fragilidades de segurança e integridade de dependências.

Inspirado no conceito de *code smells*, os termos refere-se a indícios que indicam potenciais problemas no desenvolvimento e na distribuição de software. Estes indícios são descritos de forma diferente por vários autores, incluindo *software supply chain smells*, *dependency smells* ou *weak link signals*. Apesar das diferenças terminológicas, estes conceitos referem-se ao mesmo tipo de fenómeno: *smells* associados a dependências de terceiros em sistemas de software, particularmente em aplicações web como React.

Tal como os *code smells*, estes *smells* não representam vulnerabilidades, ataques ou falhas exploradas, mas funcionam como sinais precoces de possíveis problemas. Quando ignorados, podem evoluir para vulnerabilidades críticas, comprometendo a segurança e a manutenção do sistema.

A entrega de software, na atualidade, depende fortemente de componentes externos, como bibliotecas e APIs de terceiros, ferramentas de integração contínua (CI/CD) e infraestruturas em cloud. A presença de *smells* ou indícios pode comprometer a integridade do sistema.

O conceito semelhante *dependency smells* refere-se ao mesmo tipo de problemas mas de forma mais abrangente (ex.: dependências não usadas, dependências com versões erradas, etc).

O conceito de *Software Supply Chain Security* (SoftwareSeni, 2026), inclui *registry*, repositório e *build*, que consiste no conjunto de práticas, processos e tecnologias destinadas a proteger:

- Integridade (software não é alterado indevidamente);
- Confidencialidade (dados e credenciais não são expostos);
- Disponibilidade (serviços e componentes permanecem operacionais).

Weak link signals surge em trabalhos como uma forma de alerta/indicador focado em pontos frágeis na cadeia que podem ou não representar risco futuro.

Estes termos em projetos React referem-se essencialmente ao mesmo tipo de problema que é sinais de risco ou má qualidade nas dependências de um projeto que poderão vir a tornar-se vulnerabilidades reais.

2.4.2 Tipos e Categorias

Os *smells* podem ser organizados em categorias diferentes e para cada trabalho/artigo, cada *smell* possui designações diferentes. Surge assim a necessidade de uniformizar os conceitos e terminologias de todos os *smells* abordados com a criação da tabela 2.1 com o nome dos *smells*, descrição e tipo:

Tabela 2.1: Smells detetados por Dirty Waters, Depcheck, DependencySniffer e Snyk

Smell	Descrição	Tipo
Source code inacessível	Código-fonte de uma dependência não está disponível ou acessível publicamente, dificultando a sua análise, auditoria e verificação.	Confiabilidade
Tag inacessível	Versão referenciada não é localizada no repositório.	Gestão de Dependências
Pacote Obsoleto (Deprecated Package)	Uso de dependências descontinuidas ou sem suporte ativo.	Manutenção
Fork ou URL dependency	Dependência obtida de repositórios externos ou <i>forks</i> .	Segurança
Proveniência inválida	Falta de garantia de origem ou integridade do pacote.	Segurança
Dependência não utilizada	Dependência instalada mas não usada no código.	Manutenção
Dependência não declarada	Dependência usada mas não listada no manifest.	Manutenção
Dependência em falta	Biblioteca necessária mas não instalada.	Manutenção
Bloated dependencies	Excesso de dependências desnecessárias.	Manutenção
Pinned dependency	Versão fixa sem flexibilidade de atualização.	Gestão de Dependências
Restrictive dependency	Restrições demasiado rígidas de versão.	Gestão de Dependências
Permissive dependency	Restrições demasiado abertas de versão.	Gestão de Dependências

Smell	Descrição	Tipo
Missing package-lock	Ausência de lockfile, afetando reprodutibilidade.	Configuração
Active scripts in install	Execução de scripts durante instalação com risco potencial.	Segurança
License anomalies	Conflitos de licenças entre dependências.	Segurança
Transitive dependencies issues	Problemas em dependências transitivas.	Manutenção
Too many maintainers	Excesso de mantenedores com permissões elevadas.	Segurança
Missing 2FA	Falta de autenticação de dois fatores.	Segurança
Insecure CI/CD	Pipeline de integração/deploy insegura.	Segurança

2.4.3 Relação a trabalhos já apresentados

Os conceitos de *Software Supply Chain Smells* (Liu et al., 2025), *Dependency Smells* ou *Weak Link Signals* estão relacionado a várias linhas de investigação e trabalhos, mas referem-se essencialmente ao mesmo aspeto. O termo *Software Supply Chain Smells* aparece nos trabalhos: "Dirty-Waters: Detecting Software Supply Chain Smells" e "Software Supply Chain Smells: Lightweight Analysis for Secure Dependency Management".

O conceito de *Dependency Smells* aparece em trabalhos: "Dirty-Waters: Detecting Software Supply Chain Smells", "Dependency Smells in JavaScript Projects" e "Software Supply Chain Smells: Lightweight Analysis for Secure Dependency Management".

E o conceito de *Weak Link Signals* aparece no trabalho: "What are Weak Links in the npm Supply Chain?".

Estes trabalhos estão relacionados ao redor do mesmo tipo de problema, que é descrever indícios de risco na cadeia de fornecimento de *Software*.

Capítulo 3

Análise e Conceção da solução

3.1 Contexto

A falta de ferramentas no mercado que permitam avaliar e analisar o risco de aplicações web baseadas em React surge como forte motivação para a criação desta ferramenta. Apesar da existência de ferramentas que permitem detetar problemas de segurança nos pacotes de terceiros, o que não é o objetivo deste trabalho, esta ferramenta foi desenhada e analisada para detetar (*smells*). A forte utilização de dependências externas aceleram o desenvolvimento destas aplicações, promovendo a reutilização de código e a implementação mais eficiente através de mecanismos já usados, mas em contrapartida, esta prática constitui desafios ao nível da cadeia de fornecimento de software (*Software Supply Chain*), uma vez que cada dependência, muitas vezes não estudada com profundidade, representa um potencial de risco para a aplicação. As aplicações React, com o aumento e complexidade dos projetos, tendem a acumular um elevado número de dependências, que se não forem devidamente analisadas podem originar diversos problemas de segurança e eficiência.

Estes problemas designados como *Software Supply Chain Smells* não representam necessariamente falhas e problemas da aplicação, mas constituem indicadores de risco que podem comprometer a segurança, manutenibilidade e eficiência do projeto. Assim, alguns dos principais riscos destacam-se:

- Uso de dependências desnecessárias;
- Bibliotecas sem manutenção ativa;
- Múltiplas versões da mesma dependência;
- Crescimento da árvore de dependências o que aumenta a superfície de ataque;
- Introdução indireta de vulnerabilidades.

Atualmente existem algumas ferramentas, abordadas nos capítulos anteriores, como por exemplo, ferramentas para fazer a análise do risco das dependências, Dirty Waters, Depcheck e Snyk.

Apesar de úteis estas ferramentas apresenta algumas limitações, como, a ferramenta Dirty Waters não deteta *bloated dependencies*, a incapacidade de detetar certos *smells* como dependências duplicadas ou excesso de complexidade e a ferramenta Depcheck encontra apenas dependências não utilizadas e dependências em falta. A ferramenta DependencySniffer é usada para analisar como o sistema está estruturado e ligado pelas dependências. A ferramenta Snyk por sua vez deteta e analisa *smells* de segurança como possíveis instalações de scripts maliciosos e verificação das licenças nos diretórios das dependências.

Surge assim a necessidade de criar esta ferramenta chamada Dependency Risk Analyser capaz de:

- Analisar automaticamente dependências de projetos React;
- Integrar e completar ferramentas existentes;
- Identificar (*smells*);
- Avaliar o impacto das dependências a nível global no projeto;
- Focar-se na análise preventiva da aplicação.

A ferramenta pretende ler ficheiros, como, *package.json* de outros projetos e com o auxílio de ferramentas de análise de dependências, como, Dirty Waters, Depcheck, DependencySniffer, Snyk, produzir um relatório com os *smells* e uma avaliação global do estado das dependências, bem como, o uso da ferramenta Syft para gerar o SBOM. Esta ferramenta surge como um complemento a ferramentas existentes fornecendo uma perspetiva voltada para a qualidade e sustentabilidade do software.

3.2 Domínio do problema

Esta secção pretende especificar conceitos de domínio do problema. O domínio do problema insere-se na análise da cadeia de fornecimento de software, em particular, em projetos React. Este domínio envolve a gestão de dependências externas, avaliação da qualidade e identificação de potenciais riscos.

3.2.1 Modelo Domínio

A figura 3.1 mostra o Modelo Domínio detalhado com os termos relevantes e os atributos do domínio do problema:

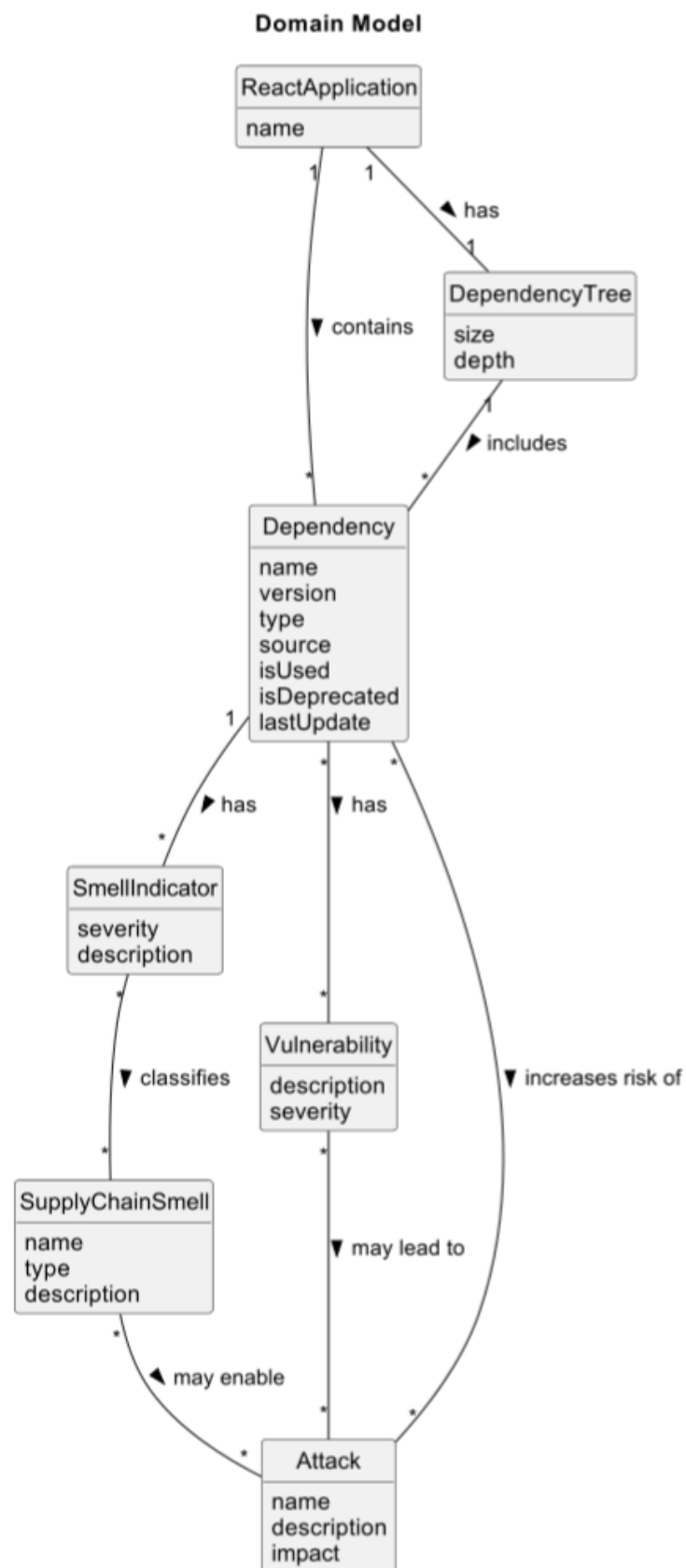


Figura 3.1: Modelo Domínio

3.2.2 Termos do domínio

- **Attack**: representa um ataque à cadeia de fornecimento de software que pode ocorrer devido a vulnerabilidades ou más práticas nas dependências.
- **Dependency**: representa o termo "dependência" que é utilizado pela aplicação.
- **DependencyTree**: representa a estrutura hierárquica das dependências do projeto que permite identificar as relações entre as dependências.
- **ReactApplication**: representa a aplicação React analisada, onde estão definidas e geridas as dependências do projeto.
- **Report**: refere-se ao relatório gerado com a informação dos *smells* nas dependências.
- **SmellIndicator**: refere-se a um indicador individual de risco numa dependência.
- **SupplyChainSmell**: padrão ou possível problema que poderá vir a surgir na aplicação React.
- **Vulnerability**: representa uma vulnerabilidade de segurança numa dependência, que pode ser explorada num possível ataque.

3.2.3 Relações e Associações

- **ReactApplication** contém **Dependency**;
- **ReactApplication** possui **DependencyTree**;
- **DependencyTree** inclui **Dependency**;
- **Dependency** possui **SmellIndicator**;
- **SmellIndicator** classifica **SupplyChainSmell**;
- **Dependency** possui **Vulnerability**;
- **Dependency** aumenta o risco de **Attack**;
- **SupplyChainSmell** pode permitir **Attack**;
- **Vulnerability** pode levar a **Attack**;

3.2.4 Glossário

A tabela 3.1 mostra o Glossário com os termos que envolvem o domínio e respetiva descrição:

Termo	Descrição
Aplicação React	Aplicação web desenvolvida com a biblioteca de JavaScript. É baseada em componentes reutilizáveis para construir interfaces de utilizador.
Árvore de Dependências	Estrutura que contém e representa todas as dependências de um projeto.
Avaliação de Risco	Análise e clarificação dos riscos associados às dependências de um projeto.
Cadeia de fornecimento de Software (Software Supply Chain)	Conjunto de dependências, componentes e processos que estão no desenvolvimento de uma aplicação.
Complexidade de Dependências	Medida para determinar o número e profundidade do conjunto de dependências de um projeto.
Dependência	Biblioteca externa utilizada por uma aplicação para fornecer funcionalidades específicas, declarada no ficheiro package.json.
Dependência Desatualizada	Biblioteca cuja versão já não é a mais recente.
Dependência Direta	Dependência explicitamente declarada pelo programador.
Dependência Duplicada	Múltiplas versões da mesma biblioteca encontram-se no projeto.
Dependência Não Utilizada	Biblioteca declarada no projeto que não é usada na aplicação.
Dependência Sem manutenção	Biblioteca que não recebe atualizações por muito tempo. Pode indicar abandono.
Dependência Transitiva	Dependência que não é declarada diretamente pelo programador. É incluída através de outra dependência.
Indicador de Risco	Sinal que pode sugerir a existência de um problema ou vulnerabilidade futura.
Software Supply Chain Smell	Indicador de potencial problema na cadeia de dependências Podem ser dependências desatualizadas, sem manutenção, dependências desnecessárias e dependências com risco conhecido.
Superfície de Ataque	Área coberta ou ponto onde o sistema pode sofrer um ataque e ser explorado. Aumenta com o número de dependências.
Vulnerabilidade	Falha de segurança que pode ser explorada na aplicação. Esta pode ser de uma dependência ou não.

Tabela 3.1: Glossário

3.3 Requisitos

Para a realização desta ferramenta foram definidos um conjunto de funcionalidades de alto nível cujo o utilizador tem acesso para realizar análise e deteção de risco de aplicações React. A funcionalidade central é permitir que a aplicação React seja analisada pela ferramenta e assim detetar *smells* que futuramente poderão vir a surgir no projeto como problemas. Para a realização desta análise, o sistema deve ser capaz de realizar um conjunto de funcionalidades menores, como, a integração de ferramentas externas, Dirty Waters, para avaliar versões e sua segurança, avaliar dependências transitivas e classificar o risco. E a ferramenta Depcheck capaz de verificar dependências não usadas e dependências usadas mas não declaradas. A ferramenta chamada DependencySniffer será adaptada para detetar possíveis problemas nas versões das dependências. E a ferramenta Snyk que surge para verificar e analisar *smells* de segurança como a instalação indevida de *scripts*, através das dependências, que podem ser perigosos e análise de licenças e a proveniência.

Uma parte importante é a capacidade do sistema conciliar a informação das ferramentas e gerar um relatório com os *smells* associados ao projeto, identificando padrões de risco.

A figura 3.2 mostra o Diagrama de Casos de Uso:

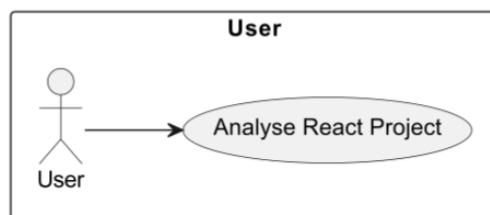


Figura 3.2: Diagrama de Casos de Uso

O modelo **ISO/IEC 25010** (ISO/IEC, 2011) é um modelo de avaliação de qualidade de software que permite determinar quais as características de qualidade de um sistema serão levados em conta. O modelo **ISO/IEC 25010** contém nove critérios de avaliação de qualidade, como, **Functional Suitability** (Qualidade funcional), **Performance Efficiency** (Performance), **Compatibility** (Compatibilidade), **Interaction Capability** (Interação), **Reliability** (Confiabilidade), **Security** (Segurança), **Maintainability** (Manutenibilidade), **Flexibility** (Flexibilidade) e **Safety** (Segurança física).

Os atributos de qualidade neste projeto são:

- **Qualidade funcional:** O sistema deve garantir que a análise de *smells* nas dependências é eficaz para cada tipo de *smell*. Deve apresentar resultados precisos com o trabalho conjuntos de todas das ferramentas de análise de risco utilizadas. Devem também se realizados vários tipos de testes desde testes unitários a testes com outras ferramentas para avaliar a qualidade da ferramenta.
- **Performance:** Deve ser garantida uma eficiência de desempenho ao executar a ferramenta, tempos de resposta que podem variar entre alguns segundos para projetos React mais pequenos, e no máximo uma hora para projetos maiores, mostrando assim o relatório da análise das dependências.

- **Compatibilidade:** O sistema deve permitir que cada ferramenta externa se ajuste para ser compatível com o conjunto da análise de dependências. Deve ser capaz de ser integrado em aplicações React permitindo a troca e leitura dos pacotes dos projetos. E também deve ser capaz de ser executada pela linha de comandos para que seja executada.
- **Interação:** O sistema deve disponibilizar uma interface clara e simples tanto para correr a ferramenta como para mostrar e indicar os dados dos relatórios gerados, como, os *smells* detetados. Deve mostrar mensagens de erro de forma clara para que o utilizador saiba com clareza o que está a falhar para assim usar a ferramenta e obter a informação o mais rápido e simples possível. Deve também mostrar ao utilizador onde o relatório foi gerado a fim de clarificar onde foi guardado para consulta e o respetivo SBOM.
- **Confiabilidade:** O sistema deve garantir a confiabilidade ao executar a análise das dependências de forma consistente e sem falhas. Deve ser capaz de lidar com erros ou falhas de forma controlada, mantendo o funcionamento esperado sempre que possível, e permitir a recuperação adequada em caso de interrupções, preservando os dados e resultados da análise.
- **Segurança:** Deve existir segurança no tratamento e recolha de todos os dados utilizados como as dependências e o relatório gerado, fornecendo apenas esses dados às pessoas certas. Para além da recolha segura da árvore das dependências, o sistema não deve conter nenhuma fonte de potencial ataque tanto para a ferramenta como para o projeto React em si.
- **Manutenibilidade:** O sistema deve ser capaz de ser aberto para que sejam adicionadas novas ferramentas de análise de outros tipos de *smells* para além dos que são abordados e testados nesta ferramenta. Deve permitir que a reutilização do SBOM (*Software Bill of Materials*) para diminuir o tempo de execução e construir um novo caso a árvore das dependências seja atualizada. Deve também conter teste que assegurem a validação contínua das várias funcionalidades.
- **Flexibilidade:** O sistema deve garantir a flexibilidade de execução em diferentes ambientes, tanto para projetos React de grande extensão, com para projetos mais curtos. Deve ser capaz de lidar também com diferentes comprimentos e larguras de árvores de dependências, sem alterar de forma drástica o desempenho. Deve proporcionar uma execução em projetos React sem grandes impactos no projeto em si.
- **Segurança física:** O sistema deve garantir a segurança operacional ao evitar que situações de risco nas dependências comprometam o funcionamento da aplicação ou a integridade do sistema. Deve restringir automaticamente a execução de análises ou ações quando forem detetadas condições potencialmente perigosas. Deve emitir alertas sobre potenciais problemas, como erros de execução, garantir que a integração de novas ferramentas de análise não compromete a segurança do sistema.

3.4 Conceção

Esta secção pretende mostrar como todo o projeto foi concebido e cada uma das partes envolventes, desde ferramentas que são capazes de cobrir determinado *smell* e escolha das ferramentas e *smells*, mas também apresentar a arquitetura global da solução, bem como toda a justificação em relação para a arquitetura usada.

A tabela 3.2 mostra para cada *smell* a ou as ferramentas capazes de o detetar:

Tabela 3.2: Ferramentas existentes que detetam cada um dos *smells*

Smell	Ferramenta
Source code inacessível	Dirty Waters, OpenSSF Scorecard
Tag inacessível	Dirty Waters
Pacote obsoleto (deprecated)	Dirty Waters, Snyk
Fork ou URL dependency	Dirty Waters
Proveniência ou assinatura inválida	Dirty Waters, Snyk, OpenSSF Scorecard
Dependência apelidada	Dirty Waters
Dependência não utilizada	Depcheck, Knip
Dependência não declarada	Depcheck, Knip
Dependência em falta	Depcheck
Bloated dependencies (Dependências inchadas)	Depcheck, DependencySniffer
Pinned dependency (Dependência fixada)	DependencySniffer, Snyk
Restrictive dependency (Dependência restritiva)	DependencySniffer, Snyk
Permissive dependency (Dependência permissiva)	DependencySniffer, Snyk
Falta de package-lock / árvore de dependências	DependencySniffer
Active install scripts (Instalação de Scripts ativos)	Snyk
License anomalies (Anomalias na licença)	Snyk
Transitive dependencies (Dependências transitivas)	Snyk, Knip
Too many contributors / maintainers (Muitos contribuidores/mantenedores)	OpenSSF Scorecard
Falta de 2FA	OpenSSF Scorecard
CI/CD Inseguro	OpenSSF Scorecard

Com base na tabela anterior e em busca a alcançar uma melhor eficiência da ferramenta global e comunicação entre as ferramentas utilizadas que ficam responsáveis por detetar os *smells* que o projeto é capaz de detetar estão descritos na tabela 3.3:

Tabela 3.3: Ferramentas e *smells* explorados no projeto

Smell	Ferramenta
Source code inacessível	Dirty Waters
Tag inacessível	Dirty Waters
Pacote obsoleto (deprecated)	Dirty Waters
Fork / URL dependency	Dirty Waters
Proveniência inválida / assinatura inválida	Dirty Waters
Dependência apelidada	Dirty Waters
Dependência não utilizada	Depcheck
Dependência não declarada	Depcheck
Dependência em falta	Depcheck
Bloated dependencies	Depcheck
Pinned dependency	DependencySniffer
Restrictive dependency	DependencySniffer
Permissive dependency	DependencySniffer
Falta de package-lock / árvore de dependências	DependencySniffer
Scripts ativos na instalação	Snyk
Anomalias de licença	Snyk
Dependências transitivas vulneráveis	Snyk

3.4.1 Visão Geral da Arquitectura

Foram abordadas duas iterações de solução, avaliados os aspetos relevantes e foi feita para cada uma delas uma avaliação crítica, sendo adotada a segunda iteração. Para isso serão utilizados os modelos C4 (Brown, 2023) e 4+1 (Kruchten, 1995), que são modelos que permitem descrever o sistema de forma estruturada e que usam diferentes níveis de abstração e vistas complementares. É mostrado o que é comum às duas iterações e depois o *design* de cada uma das iterações separadamente.

O modelo C4 permite especificar quatro níveis desde uma visão geral do sistema até um nível mais detalhado.

- O Nível 1 mostra o sistema como um todo e o seu contexto.
- O Nível 2 representa como o sistema é dividido em contentores, descrevendo quais são as principais partes do sistema.
- O Nível 3 descreve o interior de cada contentor mostrando os componentes e lógicas internas.
- O Nível 4 mostra os detalhes da implementação e como o código é feito.

O modelo 4+1 mostra a arquitetura através de diferentes perspetivas: vista lógica, vista de física, vista de processos e vista de implementação. A combinação destes dois modelos permite combinar as diferentes vistas com os diferentes níveis de abstrações devidamente separadas.

Vista Lógica

A vista lógica tem como objetivo mostrar como as responsabilidades do sistema estão distribuídas, garantindo a separação de preocupações e facilita a manutenção do sistema.

As figuras 3.3, 3.4 e 3.5 mostram a vista de lógica para cada um dos níveis do modelo C4, do nível 1 ao 3.

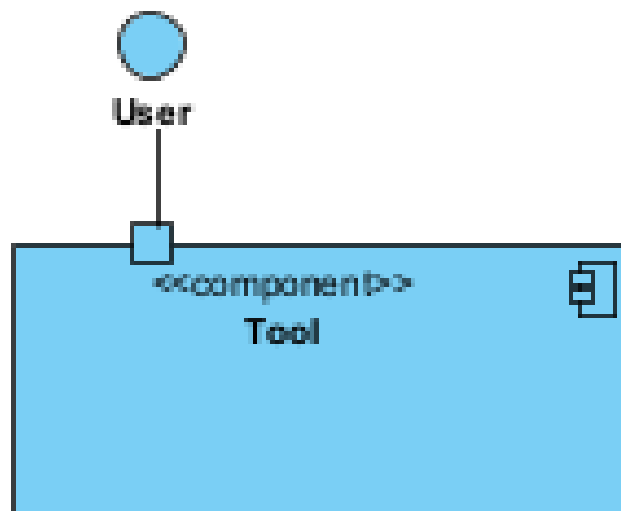


Figura 3.3: Vista Lógica Nível 1

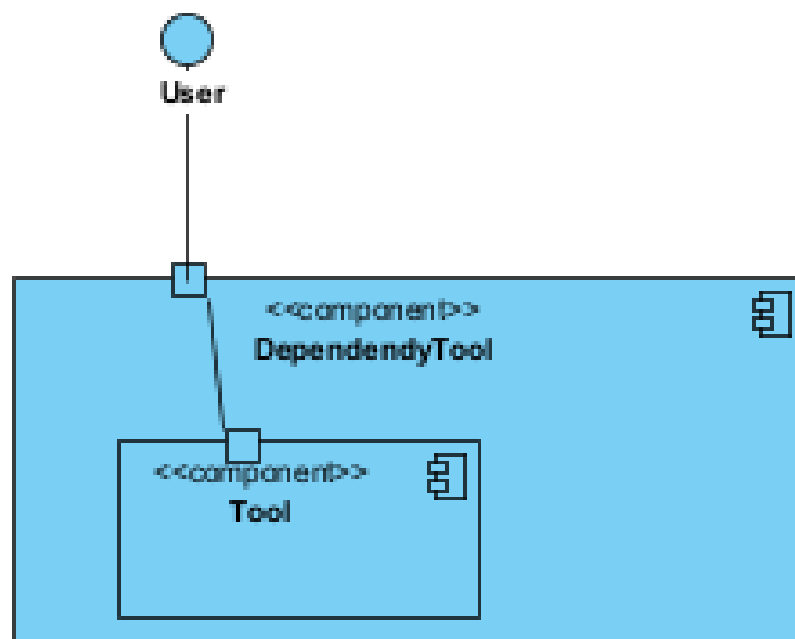


Figura 3.4: Vista Lógica Nível 2

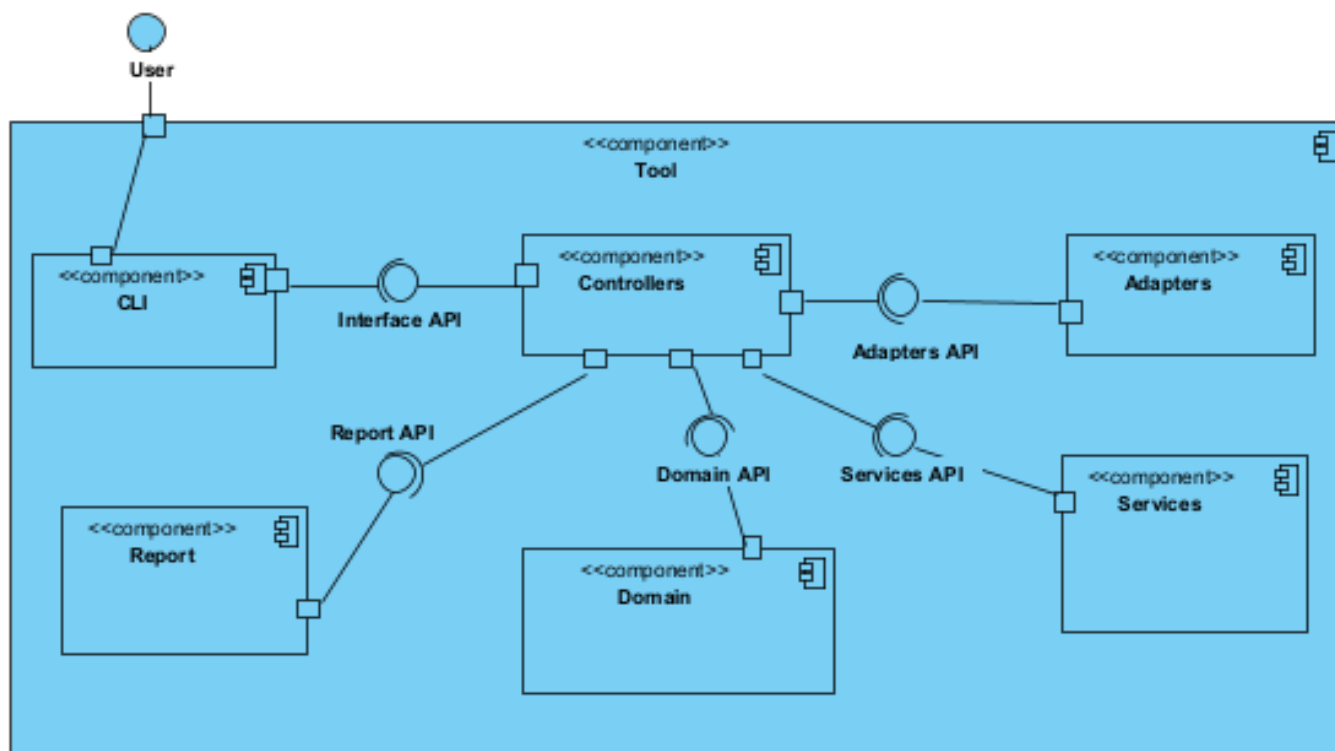


Figura 3.5: Vista Lógica Nível 3

Vista Física

Relativamente à vista física, ela mostra todos os elementos físicos e como o sistema está implementado em termos de infraestrutura. A figura 3.6 mostra a vista física que contém os diferentes componentes de software.

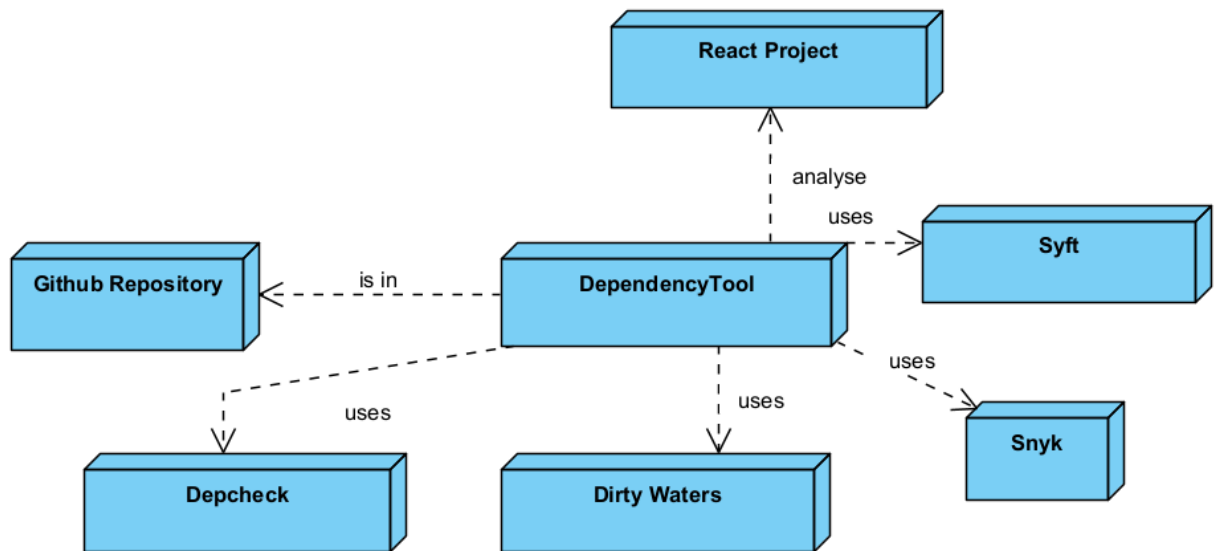


Figura 3.6: Vista Física

Vista de Processos

Esta vista foca-se no comportamento e comunicação entre as diferentes partes do sistema e mostra o fluxo dos processos e operações que ocorrem. Para isto foram feitos diagramas de sequência de sistema, e diagramas de sequência, para assim, mostrar como o utilizador interface com a ferramenta pela parte da execução da ferramenta para a análise do projeto React.

A figura 3.7 mostra o diagrama de sequência de sistema da análise dos *smells*:

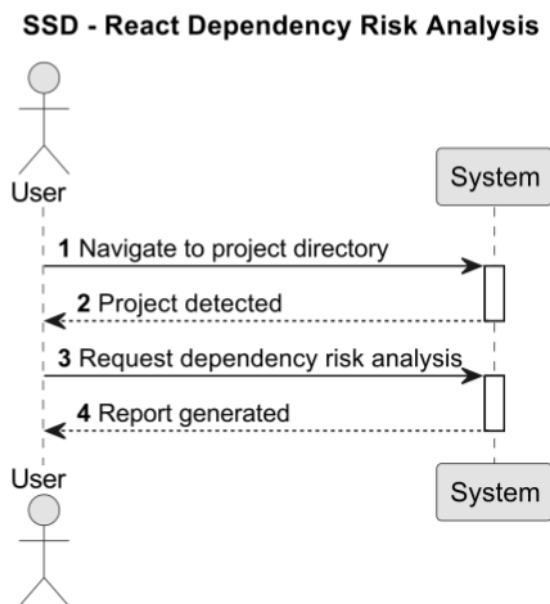


Figura 3.7: Vista de Processos Nível 1 - Análise dos *smells*

O *Node Package Manager* (npm) é usado para instalar e gerir dependências para, depois o sistema estar preparado para executar a ferramenta com todos os pré-requisitos. (Node.js, 2022)

Por ser uma interface externa, ela não é detalhada nos diagramas, sendo como uma caixa preta, responsável pela invocação e execução dos comandos disponibilizados pela ferramenta. A criação destes comandos é feita através da definição da classe *CLI.py*, onde é estabelecido o mapeamento entre o nome do comando as classes que contêm a lógica principal da ferramenta, permitindo a sua execução direta no terminal para assim a ferramenta ser usada em ambiente Windows.

O utilizador pode executar o comando principal para iniciar o processo de análise. Este comando invoca o fluxo interno da aplicação, onde o componente *CLI* interpreta a entrada do utilizador e o *Controller* encaminha o pedido para o serviço responsável por realizar a análise.

No final da execução, o sistema agrega todos os resultados obtidos, combina os *smells* e gera um relatório em formato Markdown (.md), permitindo uma leitura simples e compatível com diversas plataformas. O ficheiro resultante contém a listagem das dependências analisadas, os respetivos níveis de risco, os *smells* identificados e outras métricas relevantes, proporcionando uma visão global do estado do projeto.

Comando utilizado para executar a análise e gerar o relatório:

```
dependencyTool analyze GitHubproject --path "ProjectPath"
```

Vista de Implementação

Na vista de implementação é mostrado como os diferentes módulos, componentes e dependências estão organizados. As figuras 3.8 e 3.9 mostram a vista de implementação nível 2 e nível 3, partindo dos componentes mais simples em direção a uma descrição mais detalhada de cada um.

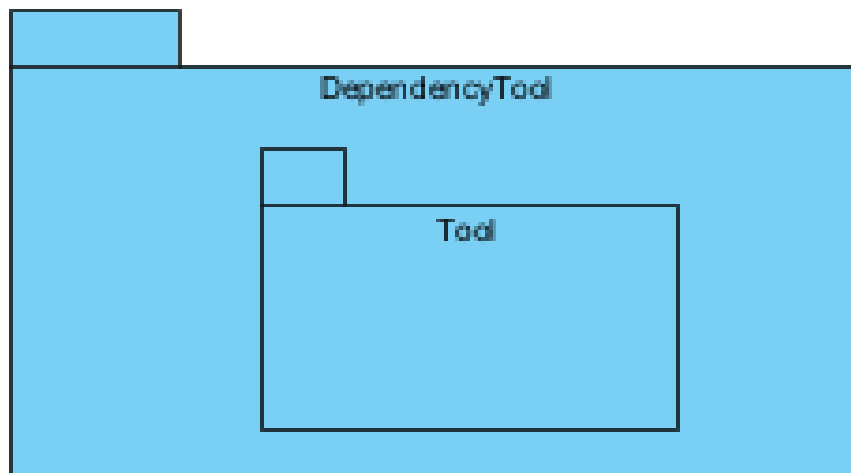


Figura 3.8: Vista de Implementação Nível 2

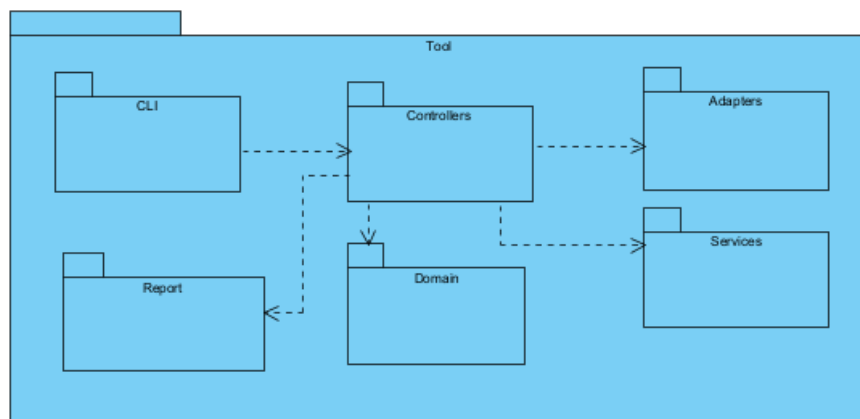


Figura 3.9: Vista de Implementação Nível 3

3.4.2 Iteração 1 — Solução Base

Esta iteração 1 foi a primeira solução proposta para a execução de uma ferramenta para detetar *smells*. Nesta iteração é mostrado o *design* da solução base, padrões aplicados, são mostrados aspetos relevantes desta arquitetura e é feita uma avaliação crítica.

Design

Para complementar a secção anterior, são mostrados os diagramas de sequência (figura 3.10) e o diagramas de classe (figura 3.11) a fim de mostrar as diferenças entre cada uma das iterações consideradas.

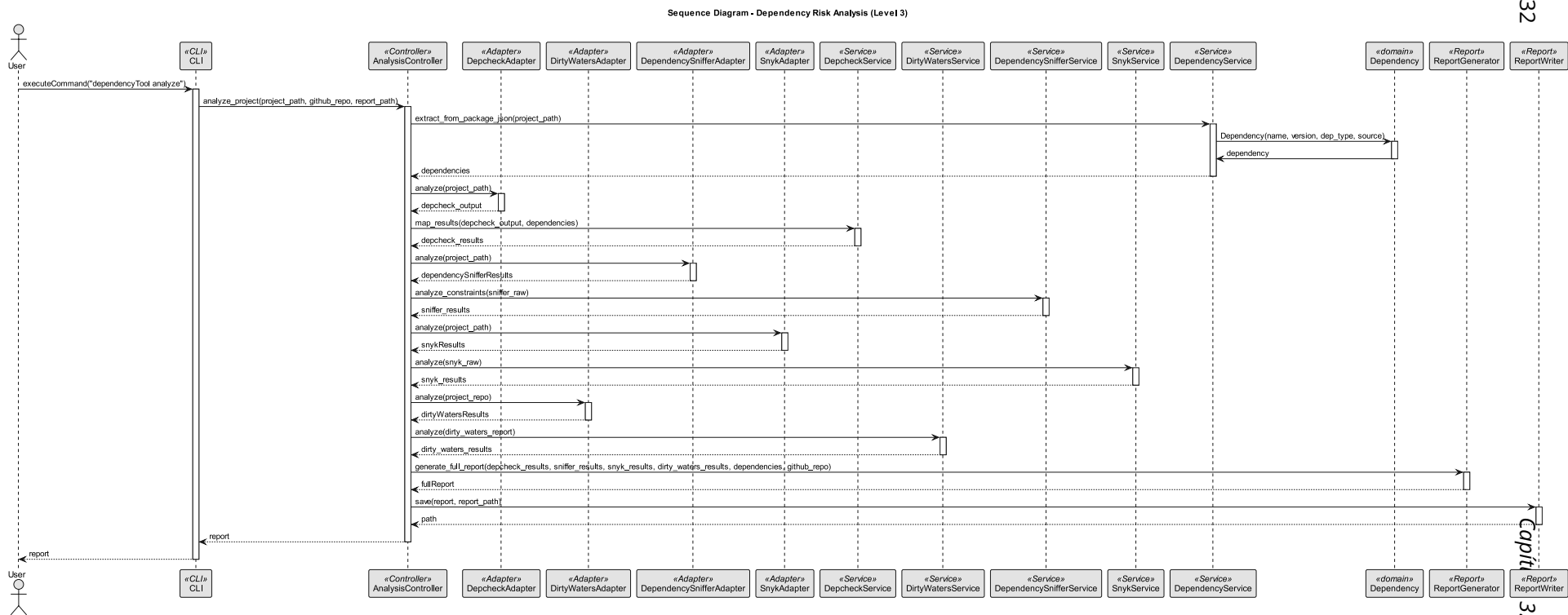


Figura 3.10: Vista de Processos Nível 3 - Diagrama de sequência da iteração 1

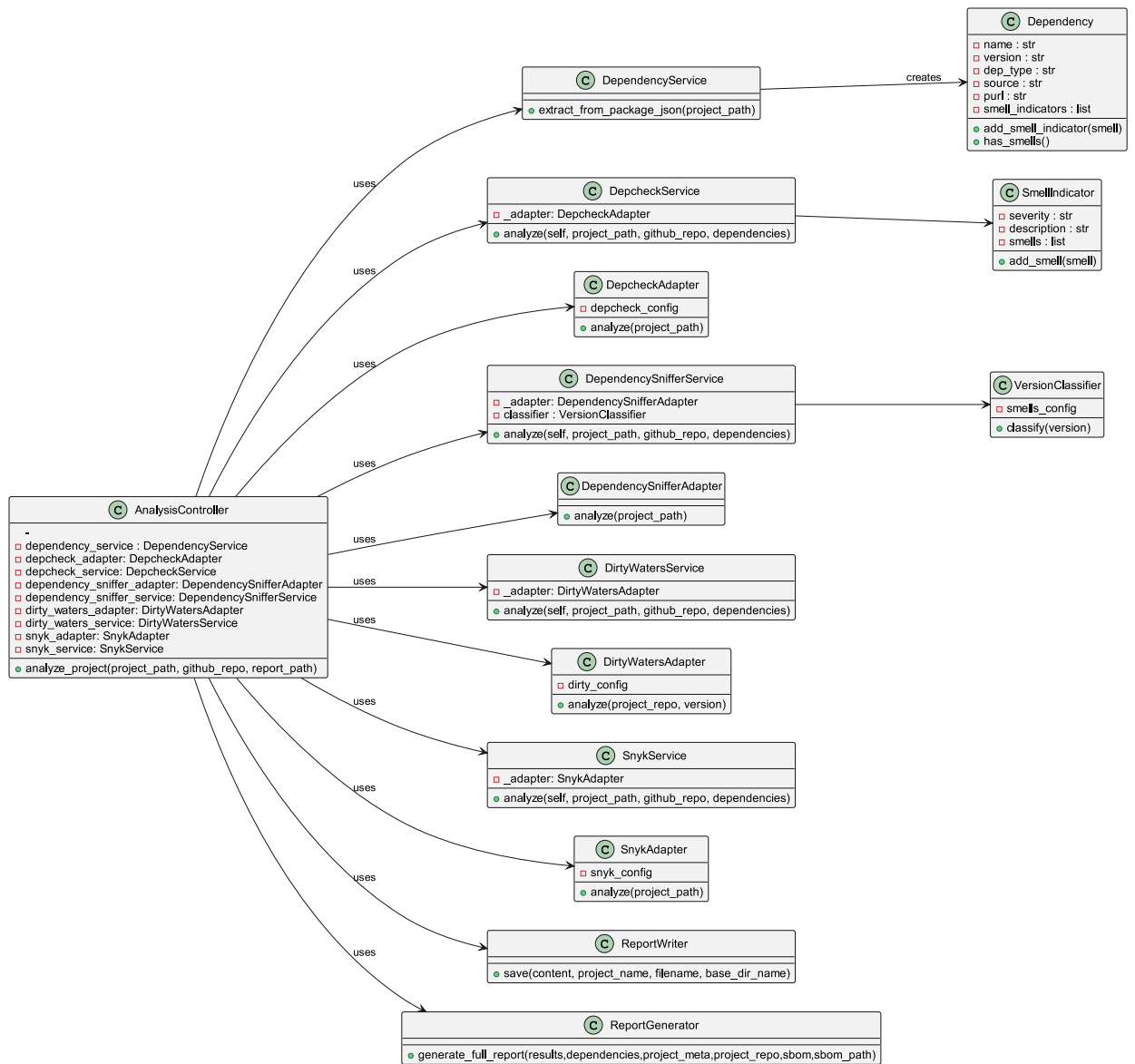


Figura 3.11: Vista de Implementação Nível 3 - diagrama de classes da iteração

Padrões de *Design* Aplicados

Os padrões aplicados nesta primeira iteração debruçam-se essencialmente em padrões abordados durante a licenciatura, com o objetivo de facilitar a manutenção e clarificação do sistema, separando responsabilidades.

O padrão **Adapter** pertence a um conjunto de padrões de design definido pelo *Gang of Four (GoF)* (DevMedia, n.d.), que permite compatibilizar interfaces distintas, possibilitando que elas trabalhem em conjunto. No contexto da ferramenta desenvolvida, o padrão Adapter foi utilizado para executar cada uma das ferramentas externas utilizadas, como Depcheck, Snyk e Dirty Waters. Cada adaptador é responsável por executar a ferramenta correspondente e converter os seus resultados para uma estrutura de dados uniforme e adequada ao restante sistema e assim serem utilizados pelos serviços para filtrar os *smells* correspondentes.

Foram usados padrões **GRASP** (*General Responsibility Assignment Software Patterns*) que descreve um conjunto de princípios que dizem quem deve ser responsável pelo quê:

- O padrão *Controller*: usado na classe *AnalysisController* onde recebe os pedidos da classe *CLI* e coordena o sistema sem conter demasiada lógica de negócio;
- O padrão *Information Expert*: usado em classes como, *Dependency* contém diretamente toda a informação sobre uma dependência e a classe *VersionClassifier* que contém toda a informação necessária para classificar versões no projeto;
- Alta coesão e baixo acoplamento (*Low Coupling and High Cohesion*): surge para reduzir o acoplamento entre componentes e cada classe concentra funcionalidades relacionadas entre si, como nos serviços *adapters* e classes de domínio;
- *Indirection*: onde os adapters funcionam como uma camada intermédia a aplicação e as ferramentas externas, o que reduz dependências diretas da aplicação;
- O padrão *Pure Fabrication*: surge em classes que não representam entidades reais do domínio, mas foram criadas para organizar responsabilidades e gerir o projeto, como as classes *AnalysisController*, *Adapters*, *Serviços*, *ReportGenerator* e *ReportWriter*;
- *Protected Variations*: o uso de classes *Adapters* protege o sistema contra alterações das ferramentas externas.

O padrão **Null Object** foi utilizado ao gerar os relatórios, evitar verificações constantes de valores nulos. Antes do processamento dos resultados, estruturas potencialmente inexistentes são substituídas por objetos vazios, como os resultados das ferramentas externas.

Aspectos Relevantes da Implementação

Esta implementação centra-se na separação de responsabilidades, sendo a classe *AnalysisController* responsável por gerir todo o processo desde a execução de cada uma das ferramentas externas até ser gerado o relatório com os *smells* detetados. Os adaptadores, como anteriormente referidos, são classes que servem de ponte entre a ferramenta externa e projeto, executando as ferramentas e disponibilizados os seus resultados para depois serem enviados ao *AnalysisController* que, por sua vez fornece aos serviços para filtrarem os seus *smells*.

A classe *Controller* depois envia os resultados de cada um dos serviços para gerar o relatório.

Avaliação Crítica

Apesar da utilização do padrão Adapter ter permitido uniformizar a interação com as diferentes ferramentas de análise, a solução implementada apresenta algumas limitações do ponto de vista arquitetural.

A principal limitação está relacionada a capacidade e facilidade de incrementar novas ferramentas ao projeto, com a violação de alguns princípios relacionados com o **SOLID**, como o *Open-Closed Principle* (OCP). Sempre que era necessário integrar uma nova ferramenta de análise, era necessário modificar a classe *AnalysisController*, adicionando novas referências ao adaptador correspondente e novas chamadas ao respetivo serviço. Desta forma, o sistema não se encontrava fechado para modificação, uma vez que a sua evolução implicava alterações em componentes já existentes.

Adicionalmente, verificava-se um elevado acoplamento entre o controlador e as ferramentas concretas. A classe *AnalysisController* possuía conhecimento explícito dos diferentes adaptadores e serviços utilizados no processo de análise, tornando-se responsável pela coordenação detalhada de todas as etapas de execução. Esta dependência dificultava a manutenção da solução.

Tabela 3.4: Comparação da integração de ferramentas com e sem o padrão Adapter

Aspeto	Sem Adapter	Com Adapter
Integração de ferramentas externas	Cada ferramenta é chamada diretamente com lógica específica	Cada ferramenta é encapsulada num adaptador próprio
Tratamento de formatos de saída	Diferente para cada ferramenta, sem normalização	Normalização dos resultados para um formato comum
Complexidade do <i>Controller</i>	Elevada, com lógica específica de cada ferramenta	Reduzida, apenas coordenação das análises
Reutilização de código	Baixa, código duplicado para integração de ferramentas	Elevada, lógica reutilizável nos adaptadores
Gestão de erros	Tratamento disperso por ferramenta	Centralizado dentro dos adaptadores
Acoplamento às ferramentas	Elevado (dependência direta de comandos e APIs)	Reduzido (dependência apenas dos <i>adapters</i>)
Manutenção	Difícil de escalar com novas ferramentas	Mais simples e modular

3.4.3 Iteração 2 — Arquitetura Extensível

A iteração 2 foi desenvolvida por se verificar limitações na capacidade do sistema em agregar novas ferramentas e o seu tempo de manutenção.

Motivação e Alternativas Consideradas

Com o objetivo de tornar o projeto extensível sem violar padrões importantes relacionados com a manutenção do sistema foi proposta com alterações em algumas classes.

A principal limitação identificada na primeira iteração estava relacionada com a dificuldade em adicionar novas ferramentas de análise. Sempre que uma nova ferramenta era integrada, era necessário modificar diretamente o *AnalysisController*, o que fazia aumentar o acoplamento e o esforço de manutenção do sistema.

A alternativa considerada tem como objetivo a classe *AnalysisController* não conhecer as classes adaptadores e para se adicionar uma nova ferramenta, não é necessário alterar a classe *controller*, sendo só necessário criar um *adapter* e um *service* para essa ferramenta a adicionar e referenciar os *smells* no relatório para serem mostrados.

Design

A solução que foi desenvolvida segue a iteração 2 o que torna a solução mais extensível, simplificando as classes a acrescentar sem grandes alterações nas outras classes, caso seja necessário acrescentar uma nova ferramenta.

As figuras 3.12 e 3.13 mostram o diagrama de sequência e o diagrama de classes para a iteração 2:

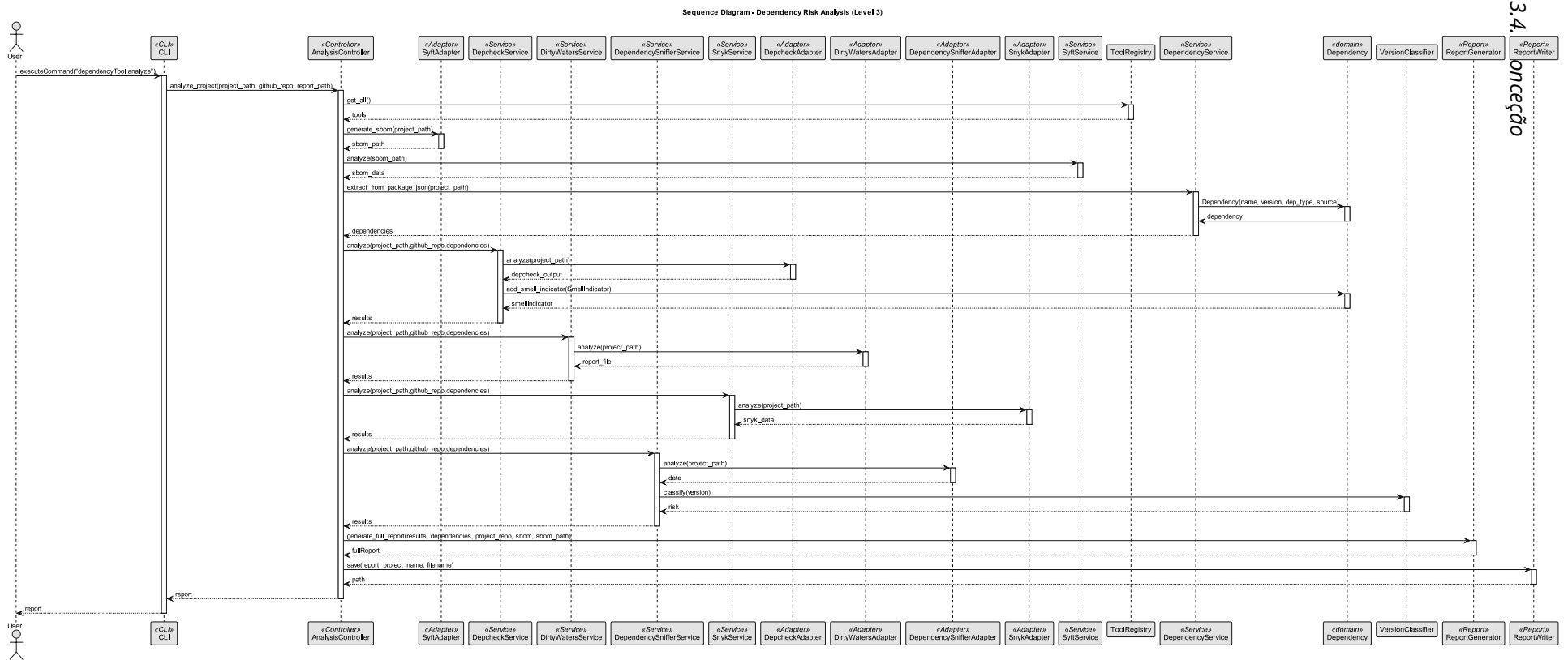


Figura 3.12: Vista de Processos Nível 3 - Diagrama de sequência da iteração 2

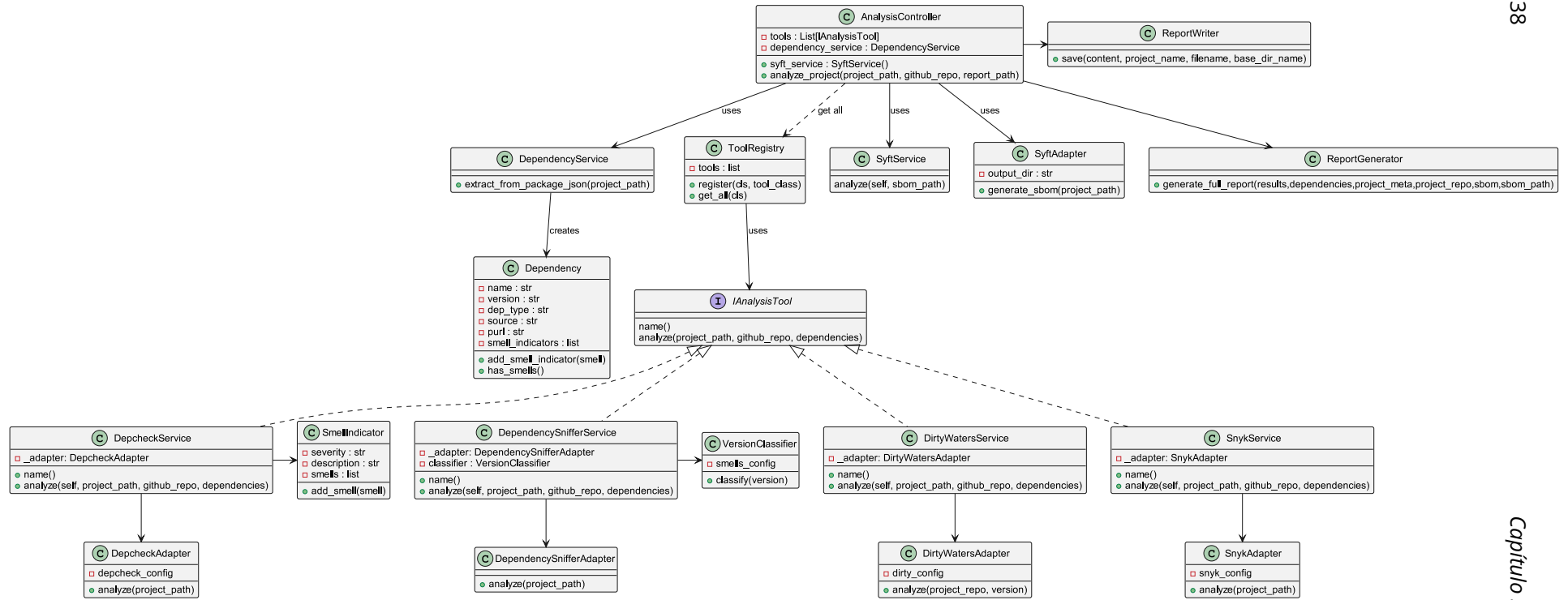


Figura 3.13: Vista de Implementação Nível 3 - diagrama de classes da Iteração 2

Padrões de Design Aplicados

Foram acrescentados alguns padrões de *design* para complementar falhas que o sistema vinha a ter, em relação, por exemplo, a violação do padrão OCP e em garantir um projeto mais extensível futuramente.

Como já referido no capítulo passado, o uso de classes *Controller*, *Services*, *Adapters*, pressupõem o uso de padrões que as sustentem e justifiquem. O uso de padrões **SOLID** para ajudarem a escrever código mais organizado, flexível e fácil de manter:

- *Single Responsibility Principle (SRP)*: usado para que cada classe tenha uma responsabilidade principal:
 - *DirtyWatersAdapter*: apenas executa e comunica com a ferramenta Dirty-Waters (geral para todas as ferramentas *Adapter*);
 - *DirtyWatersService* e outros serviços que apenas filtra e interpreta os resultados;
 - *AnalysisController* que coordena o fluxo da análise;
 - *ConfigLoader* que gere configurações;
 - *ReportGenerator* e o *ReportWriter* que servem para gerar os relatórios da análise dos *smells* dados pelas ferramentas.
- *Open/Closed Principle (OCP)*: reforça que o projeto encontra-se aberta para extensão e fechada para modificação. Caso necessário incorporar outra ferramenta cria-se um *Adapter* e um *Service* e incorpora-se no *Controller* sem necessidade de alterar significativamente a arquitetura existente.
- *Interface Segregation Principle (ISP)*: que no projeto classes, por exemplo, *Adapter* não precisam conhecer detalhes do relatório, *Services* não precisa saber como o comando é executado;
- *Dependency Inversion Principle (DIP)*: diz que a classe *AnalysisController* dependem de abstrações lógicas dos serviços e *adapters*, e não diretamente da implementação interna das ferramentas externas e as ferramentas podem ser alteradas sem modificar a lógica principal.

Foi adotado o padrão Strategy (GoF) através da criação de uma interface para os serviços das ferramentas de análise. Esta interface (*IAnalysisTool*) define um contrato que todas as ferramentas devem seguir, permitindo que cada uma implemente a sua própria lógica de forma independente. Desta forma, o comportamento do sistema passa a variar consoante a ferramenta utilizada, sem necessidade de alterações no controlador.

O padrão Registry (Fowler, 2003) foi usado para permitir que as ferramentas se registem automaticamente no sistema. Com isto, elimina-se a necessidade de instanciar as ferramentas de forma manual e direta no controlador. Esta abordagem diminui significativamente o acoplamento entre o *AnalysisController* e as implementações específicas de cada ferramenta. Basta que as ferramentas implementem a interface, facilitando a capacidade de aumentar o número de ferramentas para aumentar o número de *smells* detetados.

Foi usado também o padrão Plugin, pois permite aumentar a sistema através da adição de novos módulos independentes, sem necessidade de modificar o núcleo da aplicação. Isto permite adicionar novas ferramentas ao sistema de uma forma mais simples, diminuindo o acoplamento entre as classes e o tempo de implementação de cada ferramenta.

Aspetos Relevantes da Implementação

Um dos aspetos mais importantes levados em conta foi a separação de responsabilidades entre as camadas do sistema, nomeadamente entre o *AnalysisController*, os *Services* e os *Adapters*. Esta organização segue uma arquitetura em camadas (*Layered Architecture*), na qual cada camada possui um papel bem definido e comunica apenas com a camada adjacente, promovendo um baixo acoplamento entre componentes.

Foi implementado um mecanismo de registo dinâmico das ferramentas através do *ToolRegistry*, permitindo que cada ferramenta se registe automaticamente no sistema.

Como referido anteriormente, as ferramentas passam a implementar uma interface comum (*IAnalysisTool*), garantindo *AnalysisController* interaja com todas as ferramentas de forma abstrata.

A única ferramenta que não segue estes princípios é a ferramenta Syft porque ela não é uma ferramenta para detetar *smells*, serve para gerar, juntamente com o relatório, o SBOM de todas as dependências do projeto. Isto permite ao utilizador ter mais clareza em de como está a evoluir o seu projeto.

3.4.4 Comparação entre as Iterações

Para avaliar qual a melhor abordagem a seguir foi feita, através da tabela 3.5, uma comparação entre as duas iterações, evidenciando a evolução da arquitetura e das decisões de implementação tomadas ao longo do desenvolvimento. O objetivo é demonstrar de forma clara o impacto das melhorias introduzidas, sobretudo ao nível da extensibilidade, manutenção e acoplamento entre os diferentes componentes.

Tabela 3.5: Comparação entre as duas iterações

Aspeto	Iteração 1	Iteração 2
Integração de ferramentas	Instanciação manual no <i>AnalysisController</i>	Registo automático através de Registry
Acoplamento	Elevado acoplamento entre <i>AnalysisController</i> e ferramentas concretas	Baixo acoplamento com separação <i>Controller - Services - Adapters</i>
Extensibilidade	Adicionar novas ferramentas requer alterações no Controller	Novas ferramentas adicionadas sem alterar o <i>AnalysisController</i>
Organização da lógica	Lógica dispersa no <i>AnalysisController</i>	Lógica encapsulada em <i>Services</i> e <i>Adapters</i>
Testabilidade	Testes difíceis devido a dependências diretas	Maior facilidade de testes com isolamento de camadas
Manutenção	Maior impacto de alterações	Manutenção facilitada devido à separação de responsabilidades
Contrato entre as ferramentas (interface)	Não existe contrato formal entre ferramentas	Introdução da interface <i>IAnalysisTool</i> para uniformizar comportamento
Responsabilidades	Responsabilidades separadas em cada camada	Responsabilidades separadas em cada camada
Interação entre camadas	<i>AnalysisController</i> conhece diretamente implementações	<i>AnalysisController</i> desconhece implementações concretas (dependência por abstração)
Evolução do sistema	Estrutura mais difícil e confusa de evoluir	Estrutura simples para expansão futura

Capítulo 4

Implementação da Solução

Neste capítulo é apresentado, com base nos capítulos anteriores, a implementação da ferramenta global que conjuga várias ferramentas de análise de *smells* para assim avaliar e prevenir uma possível vulnerabilidade. É constituída por uma ferramenta chamada Syft para gerar o SBOM e três ferramentas, DirtyWaters, Depcheck, e Snyk e uma adaptação da ferramenta DependencySnnifer para detetar *smells* nas versões de cada dependência. Cada uma avalia *smells* diferentes tendo o objetivo de mostrar um relatório de deteção de *smells* completo, baseado nos *smells* abordados.

Ainda neste capítulo são mostradas cada uma das ferramentas usadas de forma individual, como ela atua e depois como é conciliada no relatório da avaliação dos *smells* do projeto.

No fim, são realizados testes utilizando vários exemplos de projetos de diferentes tamanhos e complexidades, para assim, verificar a eficiência e o tempo de execução da ferramenta como um todo.

4.1 Descrição da implementação e configurações iniciais

O projeto está organizado de forma a dividir as responsabilidades de cada ferramenta, classe e ficheiros. A solução foi desenvolvida em Python, por ser uma linguagem de alto nível, rápida e fácil de adaptar para a execução de comandos através da linha de comandos.

Como grande parte das ferramentas utilizadas já disponibiliza execução via linha de comandos, o uso da linguagem de programação Python torna a integração mais simples e eficiente. Outras vantagens do uso de Python são a facilidade do processamento de ficheiros *JSON*, relatórios *Markdown*, resultados textuais e ficheiros de configurações.

Surgem assim, no projeto, classes com responsabilidades únicas, como classes para definir configurações iniciais, classes para fazer a ligação para correr a ferramenta na linha de comandos, classes para orquestrar o fluxo do código, classes para correr as ferramentas externas utilizadas, classes para filtrar os dados das mesmas ferramentas externas, e classes de domínio. O projeto contém também ficheiros gerados pela análise das ferramentas e uma secção de documentação onde estão os diagramas considerados necessários para o entendimento e realização da solução.

O projeto foi desenvolvido com base numa arquitetura em camadas para separar responsabilidades e reduzir o acoplamento entre os diferentes componentes do sistema, permitindo assim uma melhor clareza do que cada componente faz, com melhor eficiência e diminuir o tempo de manutenção da do projeto para futuros ajustes, como a adição de novas ferramentas para detetarem outros *smells*. Segue assim, princípios de uma arquitetura

em camadas (*Layered Architecture*), onde cada camada possui responsabilidades específicas e comunica apenas com as camadas necessárias.

Foi criado um ficheiro de configuração (config.toml) que para centralização de todos os parâmetros da ferramenta, permitindo definir de forma estruturada as opções de execução, as regras de análise e os diferentes *smells* a identificar. Esta abordagem permite uma maior flexibilidade na utilização da ferramenta, uma vez que o seu comportamento pode ser alterado sem necessidade de modificar diretamente o código, promovendo assim uma melhor manutenção e escalabilidade do sistema.

O extrato de código 4.1 apresenta um excerto do ficheiro de configuração utilizado, onde se encontram definidas as ferramentas integradas, bem como os respetivos parâmetros de análise e regras de deteção de *smells*:

```
1 [tools.depcheck]
2 version_command = "npx depcheck --version"
3 analyze_command = "npx depcheck --json"
4
5 [smells.depcheck]
6 unused_severity = "medium"
7 unused_description = "Unused dependency"
8 missing_severity = "high"
9 missing_description = "Undeclared dependency"
10
11 [smells.sniffer]
12 dangerous_commands = ["rm ", "sudo ", "chmod ", "del "]
13 constraint_operators = ["^", "~", ">", "<"]
14 url_keywords = ["http", "git+"]
15
16 [tools.snyk]
17 version_command = "npx snyk --version"
18 analyze_command = "npx snyk test --json"
19
20 [tools.dirty_waters]
21 command = "dirty-waters"
22 package_manager = "npm"
23
24 check_source_code = true
25 check_source_code_sha = true
26 check_deprecated = true
27 check_forks = false
28 check_provenance = true
29 check_code_signature = false
30 check_aliased_packages = true
31
32 [licenses]
33 problematic_licenses = ["GPL", "AGPL", "UNKNOWN"]
34
35 [version_smells]
36 dangerous_versions = ["*", "latest"]
37 unstable_prefixes = ["^0.", "~0."]
38 normal_prefixes = ["^", "~"]
39
40 high_risk_label = "HIGH"
41 medium_risk_label = "MEDIUM"
42 low_risk_label = "LOW"
43 safe_label = "SAFE"
44 unknown_label = "UNKNOWN"
```

Excerto de Código 4.1: Ficheiro config.toml

Caso o utilizador queira alterar parâmetros como, adicionar licenças problemáticas ou ativar/desativar a análise de *smells* da ferramenta Dirty Waters, o ficheiro de configuração tem a função de centralizar esses parâmetros.

4.1.1 Execução por ferramenta

Para a execução de cada ferramenta foi criado um *Adapter* para correr as ferramentas externas, que são chamados pelo serviços seguindo uma arquitetura em camadas: *Analysis-Controller* - Serviços - *Adapters*.

Dirty Waters

A classe *DirtyWatersAdapter* é responsável pela integração da ferramenta Dirty-Waters onde realiza uma análise da cadeia de dependências em *smells* associados à proveniência e integridade dos pacotes. No projeto a ferramenta Dirty Waters avalia *smells* de código fonte inacessível/inválido, *tag* inacessível, dependências obsoletas (*deprecated dependencies*), *forks*, proveniência da dependência, assinatura de código inválida/falta e dependência "*Aliased*".

Este *adapter* funciona como uma camada intermédia entre a aplicação e a ferramenta externa, permitindo executar a análise através de comandos de sistema e normalizar o resultado produzido. Antes da execução, o *adapter* valida a instalação da ferramenta. Ela corre um comando que inclui o nome da ferramenta, o repositório do projeto e o *package manager* utilizado e o conjunto de *smells* a serem analisados:

```
1  command = [  
2      self.command_name ,  
3      "-p", project_repo ,  
4      "-pm", self.package_manager ,  
5  
6      "--check-source-code" ,  
7      "--check-source-code-sha" ,  
8      "--check-deprecated" ,  
9      "--check-forks" ,  
10     "--check-provenance" ,  
11     "--check-code-signature" ,  
12     "--check-aliased-packages"  
13 ]
```

Excerto de Código 4.2: Código para correr a ferramenta Dirty Waters no *DirtyWatersAdapter*

O comando vai gerar um ficheiro temporário estilo *Markdown* para depois a classe *DirtyWatersService* filtrar os *smells* necessários.

Depcheck

A classe *DepcheckAdapter* é responsável pela integração da ferramenta Depcheck utilizada para identificar dependências não utilizadas, dependências em falta e dependências inchadas (*Bloated dependencies*) no ficheiro *package.json* de um projeto. Tal como os restantes *adapters* do sistema, esta componente funciona como uma camada intermédia entre a aplicação e uma ferramenta externa.

Primeiramente o sistema verifica se o gestor de pacotes "*npm*" e a ferramenta Depcheck estão instalados corretamente com comandos que se encontram no ficheiro de configuração (para verificar a versão e executar a ferramenta Depcheck):

```
1 [tools.depcheck]
2 required_tools = ["node", "npm", "npx"]
3 version_command = "npx depcheck --version"
4 analyze_command = "npx depcheck --json"
```

Excerto de Código 4.3: Comandos para verificar a versão e executar da ferramenta Depcheck usado na classe DepcheckAdapter

A análise é realizada através da execução de um comando externo definido na configuração da aplicação, sendo este executado no diretório do projeto analisado.

```
1 result = subprocess.run(
2     self.analyze_command,
3     cwd=project_path,
4     capture_output=True,
5     text=True,
6     shell=True,
7     encoding="utf-8",
8     errors="replace"
9 )
```

Excerto de Código 4.4: Código que contém o comando para executar a ferramenta Depcheck

A dependências do "package.json" são convertidas em dependências de domínio, num objeto do tipo *Dependency* através da classe *DependencyService*, que normaliza a informação de cada dependência.

```
1 class Dependency:
2
3     def __init__(
4         self,
5         name,
6         version,
7         dep_type,
8         source,
9         purl=None
10    ):
11        self.name = name
12        self.version = version
13        self.dep_type = dep_type
14        self.source = source
15        self.purl = purl
16
17        self.smell_indicators = []
18
19    def add_smell_indicator(self, smell):
20        self.smell_indicators.append(smell)
21
22    def has_smells(self):
23        return len(self.smell_indicators) > 0
```

Excerto de Código 4.5: Classe de domínio Dependency

O resultado é capturado no formato JSON que depois na classe *DepcheckService* é filtrado com base nos *smells* que queremos observar. É verificados o JSON de output para clarificar o utilizador o mais possível onde pode estar a falhar e garantir que o sistema continua funcional mesmo perante falhas da ferramenta externa.

Snyk

A classe *SnykAdapter* é responsável pela integração da ferramenta Snyk e segue o mesmo padrão das anteriores. Apesar de poder detetar vulnerabilidades conhecidas, a utilização desta ferramenta foi exclusivamente para a deteção de *smells* de instalação de *scripts* ativos, licenças problemáticas e dependências transitivas.

Antes de executar a ferramenta, à semelhança das anteriores é verificado se a ferramenta está devidamente instalada, com mensagens claras para que o utilizador saiba onde e quando esta a falhar. A execução da ferramenta também é feita através do ficheiro de configuração:

```
1 [tools.snyk]
2 version_command = "npx snyk --version"
3 analyze_command = "npx snyk test --json"
```

Excerto de Código 4.6: Extrato no ficheiro de configuração para executar a ferramenta Snyk

Após a execução da ferramenta o resultado é capturado no formato JSON com validações do estado do output e se o JSON está vazio para que os mecanismos de tratamento de exceções e validação do output, permita lidar com falhas de instalação, execução ou formatação dos resultados errados.

```
1 result = subprocess.run(
2     self.analyze_command,
3     cwd=project_path,
4     capture_output=True,
5     text=True,
6     encoding="utf-8",
7     errors="replace",
8     shell=True
9 )
```

Excerto de Código 4.7: Código para executar a ferramenta Snyk

Como acontece nos *adaptes* das outras ferramentas, a falha em qualquer ponto faz com que o sistema mostre onde esta a ser dada a falha para que o utilizador saiba sempre o que fazer para conseguir utilizar a ferramenta o mais eficazmente possível:

```
1 check = subprocess.run(
2     self.version_command,
3     capture_output=True,
4     text=True,
5     encoding="utf-8",
6     errors="replace",
7     shell=True
8 )
9
10 if check.returncode != 0:
11     print("[SNYK] snyk is not installed or not accessible via npx.")
12     return {
13         "status": "error",
14         "tool": "snyk",
15         "message": "snyk is not available",
16         "vulnerabilities": []
17     }
```

Excerto de Código 4.8: Validação da instalação da ferramenta Snyk

DependencySniffer

A classe *DependencySnifferAdapter* não é responsável por realizar nenhum processo externo, mas sim para fazer a análise de ficheiros de configuração, nomeadamente, "package.json" e "package-lock.json". A classe começa com a leitura do ficheiro package.json, onde se encontram definidas as dependências principais do projeto.

Em seguida, é verificada a existência do ficheiro package-lock.json, onde a informação deles é passada para depois ser filtrada e tratada no *DependencySnifferService* onde são detetados *smells* relativos às versões nas dependências, desde dependências com versões fixas a a dependências com versões restrições que podem indicar uma baixa qualidade na sua manutenção.

```
1  def analyze(self, project_path: str):
2
3  package_json_path = os.path.join(project_path, "package.json")
4  lock_path = os.path.join(project_path, "package-lock.json")
5
6  has_package_json = os.path.exists(package_json_path)
7  has_package_lock = os.path.exists(lock_path)
8
9  package_json = {}
10
11  if has_package_json:
12  with open(package_json_path, "r", encoding="utf-8") as f:
13  package_json = json.load(f)
14
15  lock_data = None
16
17  if has_package_lock:
18  with open(lock_path, "r", encoding="utf-8") as f:
19  lock_data = json.load(f)
20
21  return {
22  "package_json": package_json,
23  "lock_file": lock_data,
24
25  "project_meta": {
26  "has_package_json": has_package_json,
27  "has_package_lock": has_package_lock
28  }
29  }
```

Excerto de Código 4.9: Método para recolher e verificar a existencia dos ficheiros package.json e package-lock.json

O resultado é posteriormente filtrado pelo *DependencySnifferService* para que depois possa ser gerado o relatório com estes *smells*.

Syft

Esta ferramenta segue uma abordagem diferente das anteriores, porque é uma ferramenta para gerar a lista de todas as dependências de um projeto. Por isso, esta ferramenta, no projeto, permite gerar o SBOM para que apareça no mesmo diretório do relatório, guardando o ficheiro com a data da execução para facilitar o utilizador a visualização de todas as dependências da sua aplicação.

A figura 4.1 permite observar, para os projetos "axios" e "TaskReact" os respetivos relatórios e SBOMs:

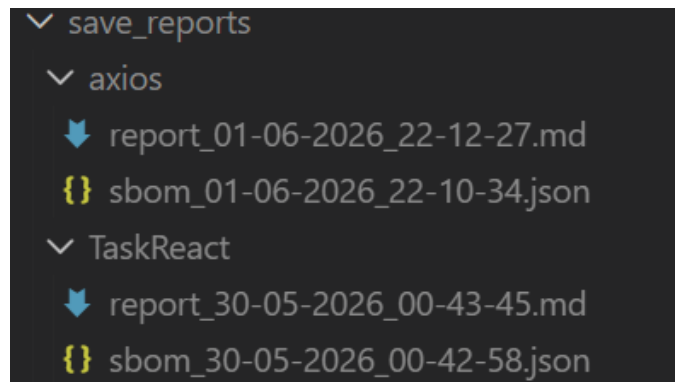


Figura 4.1: Resultados com o SBOM em formato JSON

No excerto de código 4.10 podemos observar a definição do nome do ficheiro com o SBOM e comando para executar a ferramenta Syft:

```

1  timestamp = datetime.now().strftime("%d-%m-%Y_%H-%M-%S")
2
3  sbom_file = os.path.join(
4      self.output_dir,
5      f"sbom_{timestamp}.json"
6  )
7
8  command = [
9      "syft",
10     project_path,
11     "-o",
12     "json"
13 ]

```

Excerto de Código 4.10: Comando para executar a ferramenta Syft

4.1.2 Processamento dos resultados das ferramentas

Nesta secção as classes centrais são os serviços de cada ferramenta que servem para filtrar e preparar os dados provenientes da execução de cada uma das ferramentas e assim, para que a classe *ReportGenerator* crie o ficheiro do relatório com todos os dados e *smells* analisados. Cada serviço de cada ferramenta que deteta *smells* implementa a interface *IAnalysisTool*, que define um contrato comum para todas as ferramentas do sistema. Este contrato obriga à implementação dos métodos *name()* e *analyze()*, garantindo que todas as ferramentas disponibilizam uma identificação e execução da análise. Desta forma, o controlador pode interagir com qualquer ferramenta através da mesma abstração, sem necessitar de conhecer os detalhes da sua implementação concreta.

```

1  class IAnalysisTool(ABC):
2
3      @abstractmethod
4      def name(self) -> str:
5          pass
6
7      @abstractmethod

```

```

8     def analyze(self, project_path: str, github_repo: str, dependencies:
9     list) -> dict:
        pass

```

Excerto de Código 4.11: Intergace utilizada por cada serviço de cada ferramenta

Para facilitar a extensibilidade do sistema foi introduzida a classe *ToolRegistry*. Esta classe segue o padrão *Registry*, permitindo o registo automático das ferramentas através da marcação ou decorador *@ToolRegistry.register*. Assim, quando uma nova ferramenta é adicionada ao sistema, esta fica automaticamente disponível para execução sem que seja necessário alterar o código do *AnalysisController* e sem instanciar as ferramentas no mesmo. Esta abordagem reduz o acoplamento entre componentes e facilita a evolução futura da aplicação.

```

1 class ToolRegistry:
2     _tools: List[Type[IAalysisTool]] = []
3
4     @classmethod
5     def register(cls, tool_class: Type[IAalysisTool]):
6         cls._tools.append(tool_class)
7         return tool_class
8
9     @classmethod
10    def get_all(cls) -> List[IAalysisTool]:
11        return [tool_class() for tool_class in cls._tools]

```

Excerto de Código 4.12: Classe ToolRegistry que permite marcar um serviço de uma ferramenta

```

1 @ToolRegistry.register
2 class DirtyWatersService(IAalysisTool):
3     def __init__(self):
4         ***
5
6     def name(self) -> str:
7         ***
8
9     def analyze(self, project_path, github_repo, dependencies):
10        ***

```

Excerto de Código 4.13: Exemplo de um serviço com o decorador *@ToolRegistry.register*

Dirty Waters

A classe *DirtyWatersService* como anteriormente escrito é responsável por chamar a classe *DirtyWatersAdapter* para executar a ferramenta Dirty Waters e filtrar a informação do relatório gerado pela ferramenta Dirty Waters e converter os resultados relativos aos *smells*.

Nesta classe pelo texto do relatório do Dirty Waters são extraídos os valores dos textos para cada *smell* através de expressões regulares. Através do ficheiro de configuração o projeto permite ativar/desativar os *smells*. Quando um determinado tipo de verificação está desativado, o respetivo valor é forçado a zero, garantindo que esse *smells* não é considerado no cálculo final.

```

1 stats = {
2     "missing_source_code": extract_int(r"Packages with no source code URL.*
3     ?:\s*(\d+)"),
4     "repo_404": extract_int(r"Packages with repo URL that is 404.*?:\s*(\d
5     +)"),
6     "inaccessible_sha": extract_int(r"Packages with inaccessible commit SHA
7     /tag.*?:\s*(\d+)"),
8     "deprecated": extract_int(r"Packages that are deprecated.*?:\s*(\d+)"),
9     "no_code_signature": extract_int(r"Packages without code signature.*?:\
10    s*(\d+)"),
11    "invalid_code_signature": extract_int(r"Packages with invalid code
12    signature.*?:\s*(\d+)"),
13    "forks": extract_int(r"Packages that are forks.*?:\s*(\d+)"),
14    "aliased": extract_int(r"Packages that are aliased.*?:\s*(\d+)"),
15 }

```

Excerto de Código 4.14: Uso de expressões regulares para retirar os *smells* definidos

Depcheck

A classe *DepcheckService* é responsável chamar a por mapear os resultados obtidos pela ferramenta Depcheck. Utiliza uma configuração definida no ficheiro "config.toml", permitindo parametrizar a severidade e a descrição associadas a cada tipo de *smell*:

```

1 [smells.depcheck]
2 unused_severity = "medium"
3 unused_description = "Unused dependency"

```

Excerto de Código 4.15: Parametros de configuração do Depcheck

Durante a execução do do método *analyze()* é chamada a classe *DepcheckAdapter* para executar a ferramenta e são percorridos todas as dependências e avalia dependências não utilizadas/*bloated dependencies* que são identificadas quando uma dependência existente no projeto não é referenciada no código e dependências em falta que são identificadas quando o código utiliza uma dependência que não está declarada no ficheiro de configuração do projeto.

Para cada caso é associado um *SmellIndicator* à dependência correspondente, contendo a severidade e descrição definidas na configuração. Depois os dados são devolvidos para que seja gerado o relatório de análise de *smells*.

```

1 unused = (
2     depcheck_output.get("dependencies", []) +
3     depcheck_output.get("devDependencies", [])
4 ) or []
5 missing = depcheck_output.get("missing", {}) or {}
6
7 for dep in dependencies:
8
9     dep.is_used = True
10
11     if dep.name in unused:
12         dep.is_used = False
13         dep.add_smell_indicator(
14             SmellIndicator(
15                 severity=self.unused_severity,

```

```

16         description=self.unused_description
17     )
18 )
19 if dep.name in missing:
20     dep.add_smell_indicator(
21         SmellIndicator(
22             severity=self.missing_severity,
23             description=self.missing_description
24         )
25     )
26 return dependencies

```

Excerto de Código 4.16: Processamento de dependências no DepcheckService

Snyk

A classe *SnykService* é responsável por executar e analisar os resultados fornecidos pela ferramenta Snyk. Durante a execução do método *analyze()* são classificadas três categorias relacionadas com as dependências (instalação de *scripts* ativos, licenças problemáticas e dependências transitivas):

```

1 result = {
2     "install_scripts": [],
3     "license_anomalies": [],
4     "transitive_dependencies": []
5 }

```

Excerto de Código 4.17: Smells detetados pela ferramenta Snyk

Na configuração do sistema é possível definir as licenças que o utilizador considera problemáticas:

```

1 [licenses]
2 problematic_licenses = ["GPL ", "AGPL ", "UNKNOWN"]

```

Excerto de Código 4.18: Ficheiro de configuração onde são definidas as licenças problemáticas

Com os resultados dos *smells* obtidos, são devolvidos para posteriormente serem mostrados no relatório global.

DependencySniffer

A classe *DependencySnifferService* é responsável por executar a classe *DependencySnifferAdapter* e analisar as dependências declaradas no ficheiro *package.json* e *package-lock.json* de um projeto, com o objetivo de identificar diferentes padrões e potenciais riscos associados às versões e configurações utilizadas. Para cada dependência encontrada, é realizada uma classificação de risco através de um classificador de versões pela classe *VersionClassifier*, permitindo identificar versões potencialmente instáveis ou perigosas. Estas versões, comandos e classificador de versões é configurável no ficheiro *"config.toml"*:

```

1 [smells.sniffer]
2
3 dangerous_commands = ["rm ", "sudo ", "chmod ", "del "]

```

```
4 constraint_operators = ["^", "~", ">", "<"]
5 url_keywords = ["http", "git+"]
6
7 ***
8
9 unstable_prefixes = ["^0.", "~0."]
10 normal_prefixes = ["^", "~"]
11
12 high_risk_label = "HIGH"
13 medium_risk_label = "MEDIUM"
14 low_risk_label = "LOW"
15 safe_label = "SAFE"
16 unknown_label = "UNKNOWN"
```

Excerto de Código 4.19: Configurações ligadas ao DependencySniffer

Neste serviço são categorizados cinco grupos de *smells*, dependências fixadas (*pinned dependencies*), dependências instaladas a partir de um URL, dependências com restrições rígidas, (*restrict constraints*), dependências com restrições de acesso (*permission constraints*) e dependências com risco nas versões, ou por estarem como risco na sua manutenção, ou por poderem vir a tornar-se um risco de segurança.

```
1 results = {
2   "pinned": [],
3   "url_dependencies": [],
4   "restrict_constraints": [],
5   "permission_constraints": [],
6   "version_risks": []
7 }
```

Excerto de Código 4.20: Resultado para depois ser escrito o relatório de análise de dependências

Por fim, são recolhidos os *smells* na variável "results" que é retornado para ser gerado o relatório.

Syft

Por fim, e diferente dos outros serviços, este apenas analisa os dados do SBOM e avalia outras métricas que poderão ser úteis ao utilizados, como, número total de componentes, Distribuição de licenças no projeto e Componentes sem licença, enviando os dados para a classe *AnalysisController* e assim chamar as classes que criam e preenchem o relatório.

```
1 return {
2   "sbom_file": sbom_path,
3   "total_components": len(artifacts),
4   "license_distribution": distribution,
5   "no_license": no_license
6 }
```

Excerto de Código 4.21: Métricas avaliadas pela classe *SyftService*

SBOM Summary

- SBOM file: save_reports\axios\sbom_01-06-2026_23-32-57.json
- Total components: 260
- License distribution: MIT (226), CC0-1.0 (1), ISC (2), BSD-2-Clause (1), BSD-3-Clause (2)
- Components with no license: 28

Figura 4.2: Exemplo de um sumário dos dados avaliados pela ferramenta Syft no relatório

4.1.3 Relatório dos *Smells* analisados

A escrita do Relatório com o auxílio de duas classes (*ReportWriter* e *ReportGenerator*). A classe *ReportWriter* tem o papel de guardar o relatório final gerado pelo sistema dentro de um ficheiro chamado “*save_reports*” dentro de uma pasta com o nome do projeto. É criado automaticamente um nome baseado na data e hora atual, para garantir que cada relatório é único e facilitar a consulta ao utilizador. O conteúdo do relatório é escrito num ficheiro de texto no formato *Markdown* (.md), o SBOM gerado pela ferramenta Syft e o caminho completo onde o ficheiro foi guardado é apresentado ao utilizador.

```

1 class ReportWriter:
2     def save(self, content, project_name, filename=None, base_dir_name="
      save_reports"):
3
4         now = datetime.now()
5         timestamp = now.strftime("%d-%m-%Y_%H-%M-%S")
6
7         project_name = project_name.replace("/", "_")
8
9         if filename is None:
10             filename = f"report_{timestamp}.md"
11         else:
12             name, ext = os.path.splitext(filename)
13             filename = f"{name}_{timestamp}{ext}"
14
15         base_dir = os.path.abspath(
16             os.path.join(os.path.dirname(__file__), "..", "..")
17         )
18
19         project_folder = os.path.join(
20             base_dir,
21             base_dir_name,
22             project_name
23         )
24
25         os.makedirs(project_folder, exist_ok=True)
26
27         path = os.path.join(project_folder, filename)
28
29         with open(path, "w", encoding="utf-8") as f:
30             f.write(content)
31
32         print(f"\nReport saved in: {path}")
33
34     return path

```

Excerto de Código 4.22: Classe ReportWriter

A imagem seguinte apresenta alguns exemplos de ficheiros gerados com o formato `report_DIA-MES-ANO_HORA-MINUTO-SEGUNDO`, correspondente ao padrão `%d-%m-%Y_%H-%M-%S` e o SBOM referente do mesmo modo. Cada relatório referencia qual o seu SBOM.

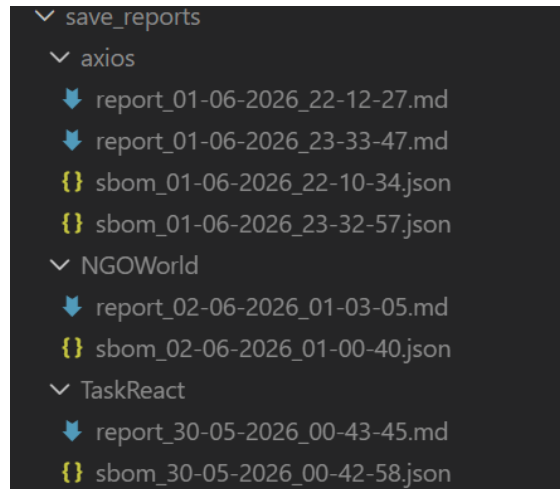


Figura 4.3: Exemplos de relatórios gerados pela ferramenta

Já a classe *ReportGenerator* é responsável por escrever no relatório da análise de dependências do projeto, reunindo e organizando os resultados de cada *smell* proveniente das diferentes ferramentas utilizadas no sistema. O método `generate_full_report()` recebe cada um dos resultados preenchidos das ferramentas utilizadas, Depcheck, Snyk, Dirty Waters e da adaptação da ferramenta *DependencySniffers* para assim, escrever no relatório.

```

1  dirty = results.get("dirty_waters", {}).get("stats", {})
2
3  md.append("\n## Other Smells Detected")
4
5  dirty_total = sum(v for v in dirty.values() if isinstance(v, int))
6
7  for k, v in dirty.items():
8      md.append(f"- {k}: {v}")
9
10 total_smells += dirty_total
11
12 md.append(f"\n### Total Smells: {total_smells}")

```

Excerto de Código 4.23: Exemplo de tratamento dos dados para escrever no relatório para a ferramenta Dirty Waters

O relatório está estruturado em várias secções, cada uma dedicada à apresentação de diferentes *smells* detetados no projeto. Primeiro são mostrados a existência ou não de ficheiros ligados à estrutura do projeto como o *"package.json"* e o *"package-lock.json"*. Seguidamente, são apresentados os *smells* relacionados com dependências, incluindo dependências não utilizadas e dependências em falta, o que permite detetar falhas na manutenção e declaração de dependências. De seguida, são apresentados *smells* relacionados com gestão e manutenção de versões, incluindo dependências com versões fixas, restrições de versão e dependências provenientes de fontes externas. Esta secção também inclui uma análise de risco associada às versões utilizadas. Posteriormente, são incluídos *smells* relacionados com segurança e licenciamento, incluindo *scripts* potencialmente perigosos, problemas de licenças e

dependências indiretas que podem introduzir riscos adicionais no projeto. São apresentados *smells* relacionados com a integridade e manutenção das dependências, como referências inválidas, código indisponível ou versões descontinuadas. Por fim, é mostrado um resumo global do número total de *smells* identificados, fornecendo uma visão geral do estado do projeto.

A figura 4.4 mostra alguns dos *smells* detetado pela ferramenta e alguns dados úteis para o utilizador:

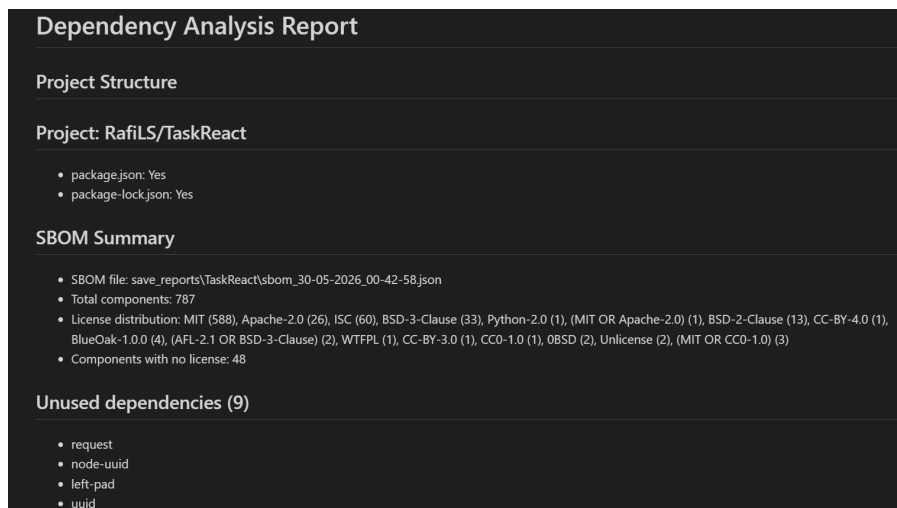


Figura 4.4: Exemplo de relatório gerado

4.1.4 Aspectos Adicionais

A ferramenta disponibiliza um comando de ajuda que permite ao utilizador consultar todas as opções e comandos disponíveis após a instalação. Este comando é uma ajuda ao utilizador para saber que comandos podem ser usados e o que a ferramenta pode proporcionar.

O comando de ajuda pode ser invocado através da opção `-h` ou `-help`. A execução deste comando não inicia qualquer análise, apenas mostra informação no terminal.

Exemplo da sua utilização podendo ser colocado o `-h` ou `-help` no final do comando para executar a ferramenta:

```
dependencyTool -h  
dependencyTool -help
```

```
usage: python.exe c:\users\rafael\appdata\roaming\python\python314\scripts\dependencyTool [-h] {analyze} ...

Dependency Analysis Tool

positional arguments:
  {analyze}
  analyze              Analyze dependencies of a project using a GitHub repo + local path

options:
  -h, --help          show this help message and exit

Usage:
  dependencyTool analyze owner/repo --path "C:\path\to\project"

Examples:
  dependencyTool analyze RafiLS/TaskReact --path "C:\Users\Rafael\Desktop\TaskReactVite"
```

Figura 4.5: Resultado da execução do comando de ajuda

Como resultado, é apresentada uma lista de comandos suportados pela ferramenta, incluindo o comando principal de análise, bem como as respectivas opções e parâmetros configuráveis.

Para correr a ferramenta é usado o comando que contém o (*owner/repo*) do projeto no GitHub e o caminho para o projeto:

```
dependencyTool analyze owner/repo --path "C:\path\to\project"
```

Exemplo de uso com o projeto real:

```
dependencyTool analyze RafiLS/TaskReact --path
"C:\Users\Rafael\Desktop\TaskReactVite"
```

```
Report saved in: C:\Users\Rafael\Desktop\Estágio\DependencyTool\save_reports\report_20-05-2026_01-36-25.md
Analysis completed successfully.
```

Figura 4.6: Execução da ferramenta na linha de comandos

4.2 Testes

Nesta secção são apresentados os diferentes tipos de testes utilizados no desenvolvimento do sistema. Os testes têm como objetivo garantir que cada componente funciona corretamente de forma isolada e em conjunto com outros componentes, para assim, assegurar a fiabilidade de cada parte do projeto. São apresentados, com exemplos, vários tipos de testes, testes unitários, testes de integração e testes de Sistema (ISTQB, 2026).

Os testes seguiram uma abordagem AAA (Arrange, Act, Assert) (Nunes, 2023), que consiste em dividir cada teste em três fases bem definidas. Na fase de *Arrange* é preparado o ambiente, incluindo a criação dos objetos, configuração e *mocks*. Na fase de *Act* é executada a funcionalidade que se pretende testar. Por fim, na fase de *Assert* são verificados os resultados obtidos, comparando-os com os valores esperados. A utilização desta abordagem permitiu melhorar a legibilidade dos testes e tornar mais fácil compreender o que está a ser testado. Além disso, facilita a manutenção do código de testes e ajuda a identificar rapidamente possíveis erros, uma vez que cada etapa do teste está claramente separada.

4.2.1 Testes Unitários

Os testes unitários surgem como uma necessidade de verificar o comportamentos de cada método e classe de forma isolada. São testes utilizados principalmente na camada de domínio, serviços e adaptadores, como por exemplo, na classe *Dependency* para validar funções como *add_smell_indicator(self, smell)* e *has_smells(self)*:

```
1 class TestDependency:
2     def test_creation(self):
3         dep = Dependency(
4             name="react",
5             version="18.2.0",
6             dep_type="prod",
7             source="package.json"
8         )
9         assert dep.name == "react"
10        assert dep.version == "18.2.0"
11        assert dep.dep_type == "prod"
12        assert dep.source == "package.json"
13        assert dep.purl is None
14        assert dep.smell_indicators == []
15
16    def test_add_smell_indicator(self):
17        dep = Dependency(
18            name="react",
19            version="18.2.0",
20            dep_type="prod",
21            source="package.json"
22        )
23        smell = SmellIndicator("HIGH", "Unused dependency")
24        dep.add_smell_indicator(smell)
25
26        assert len(dep.smell_indicators) == 1
27        assert dep.smell_indicators[0] == smell
28
29    def test_has_smells_false(self):
30        dep = Dependency(
31            name="react",
32            version="18.2.0",
33            dep_type="prod",
34            source="package.json"
35        )
36        assert dep.has_smells() is False
37
38    def test_has_smells_true(self):
39        dep = Dependency(
40            name="react",
41            version="18.2.0",
42            dep_type="prod",
43            source="package.json"
44        )
45        dep.add_smell_indicator(
46            SmellIndicator("MEDIUM", "Bloated dependency")
47        )
48        assert dep.has_smells() is True
```

Excerto de Código 4.24: Testes da classe Dependency

Cada um dos testes avalia um atributo da classe *Dependency* com os vários cenários possíveis.

Para classes *Adapter* e Serviços, como por exemplo, as classes *DirtyWatersAdapter* e *DepcheckService* foram desenvolvidos testes unitários para validar a lógica da execução das ferramentas, para os *Adapters*, e para a filtragem dos dados dos *smells* para os Serviços.

Foram utilizados *mocks* para simular estas dependências, pois são necessários recursos externos do sistema, como a execução de processos, verificação de ferramentas instaladas e leitura de ficheiros gerados. Desta forma, é possível testar diferentes cenários, tais como, a ausência da ferramenta no sistema, falhas na execução do comando e ausência de relatórios gerados.

```

1 class TestDirtyWatersAdapter:
2     def test_tool_not_installed(self):
3         with patch("shutil.which", return_value=None):
4             adapter = DirtyWatersAdapter()
5
6             result = adapter.analyze("repo/test")
7             assert result is None
8
9     def test_tool_execution_fails(self):
10        with patch("shutil.which", return_value="dirty-waters"), \
11            patch("subprocess.run") as mock_run:
12
13            mock_run.return_value = Mock(returncode=1, stderr="error")
14            adapter = DirtyWatersAdapter()
15            result = adapter.analyze("repo/test")
16
17            assert result is None
18
19    def test_no_report_generated(self):
20        with patch("shutil.which", return_value="dirty-waters"), \
21            patch("subprocess.run") as mock_run, \
22            patch("glob.glob", return_value=[]):
23
24            mock_run.return_value = Mock(returncode=0)
25            adapter = DirtyWatersAdapter()
26            result = adapter.analyze("repo/test")
27
28            assert result is None

```

Excerto de Código 4.25: Alguns testes da classe DirtyWatersAdapter

Para o *DepcheckService* bom como para todos os outros serviços, os testes feitos têm o objetivo de validar a lógica de tratamento dos dados sem depender de ferramentas externas ou chamadas a outras camadas. Focam-se principalmente na validação de diferentes cenários de entrada, que inclui casos normais, casos limite e possibilidades que podem ocorrer durante a execução real do sistema.

```

1 class TestDepcheckService:
2
3     def test_marks_unused_dependency(self):
4
5         service = DepcheckService()
6
7         deps = [FakeDependency("lodash")]
8
9         service._adapter.analyze = lambda _: {

```

```

10     "dependencies": ["lodash"],
11     "missing": []
12 }
13 result = service.analyze("/fake", "repo", deps)
14
15 dep = result[0]
16 assert dep.is_used is False
17 assert len(dep.smell_indicators) == 1
18 assert dep.smell_indicators[0].description == service.
    unused_description
19
20 def test_detects_missing_dependency(self):
21
22     service = DepcheckService()
23
24     deps = [FakeDependency("react")]
25
26     service._adapter.analyze = lambda _: {
27         "dependencies": [],
28         "missing": ["react"]
29     }
30     result = service.analyze("/fake", "repo", deps)
31
32     dep = result[0]
33     assert len(dep.smell_indicators) == 1
34     assert dep.smell_indicators[0].description == service.
        missing_description

```

Excerto de Código 4.26: Alguns testes da classe DepcheckService

Para além disso, foram testadas todas as classes do projeto de forma semelhante desde classes para mostrar informação na linha de comandos, classes, classes de configuração, a classe *AnalysisController* e classes para escrever e gerar o relatório final com a informação dos *smells* detetados pelas diferentes ferramentas.

4.2.2 Testes de Integração

Os testes de integração têm como objetivo validar a comunicação entre os diferentes componentes do sistema, garantindo que os módulos funcionam corretamente em conjunto. São combinados vários componentes, como o controlador, os serviços e os adaptadores responsáveis pela recolha e transformação de dados. Ao contrário dos testes unitários, que isolam cada componente, os testes de integração permitem verificar o fluxo completo de dados entre as diferentes camadas do sistema.

O teste realizado foca-se na execução do fluxo completo de análise, desde a extração de dependências de um projeto até gerar o relatório final. Isto é importante pois permite validar se os resultados produzidos por cada módulo são corretamente consumidos pelos seguintes.

```

1 class TestAnalysisControllerIntegration:
2     def test_full_analysis_detects_smells(self):
3
4         base_path = os.path.abspath(
5             os.path.join(os.path.dirname(__file__), "..", "test_project")
6         )
7         with patch("src.report.report_writer.ReportWriter.save") as mock_save:
8
9             controller = AnalysisController()

```

```

10
11     report = controller.analyze_project(
12         project_path=base_path,
13         github_repo="RafILS/TaskReact",
14         report_path="test_report.md"
15     )
16     assert "# Dependency Analysis Report" in report
17     assert "Project Structure" in report
18     assert (
19         "Unused dependencies" in report
20         or "Unused dependencies/Bloated Dependencies" in report)
21     assert "Missing dependencies" in report
22     assert "Version Risk Analysis" in report
23     assert "Total Smells" in report
24     assert len(report) > 100
25     assert "lodash" in report or "react" in report or "axios" in report
26
27     mock_save.assert_called_once()

```

Excerto de Código 4.27: Testes de Integração

4.2.3 Testes de Sistema

Os testes de sistema têm como objetivo validar o comportamento global da aplicação num ambiente próximo do real, garantindo que todos os componentes principais funcionam corretamente em conjunto. Os testes de Sistema focam-se na validação interna e no funcionamento global da aplicação. O sistema é executado diretamente através do controlador principal da aplicação, permitindo validar a comunicação entre as diferentes camadas do sistema de forma controlada.

No projeto, são utilizados *mocks* para simular ferramentas externas e sub processos, evitando dependências reais. Isto permite validar o fluxo completo da aplicação de forma controlada, assegurando que o sistema consegue executar todas as etapas da análise, gerar relatório final e guardar os resultados.

```

1 class TestAnalysisSystem:
2     def test_full_analysis_system_runs(self, tmp_path):
3
4         project_path = os.path.abspath(
5             os.path.join(os.path.dirname(__file__), "..", "test_project")
6         )
7         fake_report_content = """
8         Total packages in the supply chain: 10
9         Packages with no source code URL: 3
10        Packages with repo URL that is 404: 2
11        """
12
13        fake_report_file = tmp_path / "dirty_report.md"
14        fake_report_file.write_text(fake_report_content, encoding="utf-8")
15
16        with patch("src.adapters.depcheck_adapter.shutil.which", return_value="npx"), \
17             patch("src.adapters.snyk_adapter.shutil.which", return_value="npx"), \
18             patch("src.adapters.dirty_waters_adapter.shutil.which",
19                 return_value="dirty-waters"), \
20             patch("src.adapters.dirty_waters_adapter.DirtyWatersAdapter.analyze",

```

```

20         return_value=str(fake_report_file)), \
21         patch("subprocess.run"), \
22         patch("src.report.report_writer.ReportWriter.save") as mock_save:
23
24         controller = AnalysisController()
25
26         report = controller.analyze_project(
27             project_path=project_path,
28             github_repo="RafilS/TaskReact",
29             report_path="report.md"
30         )
31         assert "Dependency Analysis Report" in report
32         assert "Total Smells" in report
33         assert mock_save.called

```

Excerto de Código 4.28: Testes de Sistema

Foram realizados no total 81 testes com sucesso e executados através do comando *pytest*. A figura 4.5 mostra o resultado da execução dos testes:

```

===== test session starts =====
platform win32 -- Python 3.12.10, pytest-8.4.2, pluggy-1.6.0
rootdir: C:\Users\Rafael\Desktop\Estágio\DependencyTool
configfile: pytest.ini
testpaths: tests
collected 81 items

tests\integration\test_integration_analysis_controller.py . [ 1%]
tests\system\test_analysis_system.py . [ 2%]
tests\unit\adapters\test_depcheck_adapter.py ..... [ 8%]
tests\unit\adapters\test_dependency_sniffer_adapter.py ..... [ 14%]
tests\unit\adapters\test_dirty_waters_adapter.py ..... [ 20%]
tests\unit\adapters\test_snyk_adapter.py ..... [ 28%]
tests\unit\config\test_config_loader.py .... [ 33%]
tests\unit\controller\test_analysis_controller.py . [ 34%]
tests\unit\domain\test_dependency.py .... [ 39%]
tests\unit\domain\test_smell_indicator.py ... [ 43%]
tests\unit\report\test_report_generator.py ..... [ 54%]
tests\unit\report\test_report_writer.py .. [ 56%]
tests\unit\services\test_depcheck_service.py ..... [ 62%]
tests\unit\services\test_dependency_service.py .... [ 67%]
tests\unit\services\test_dependency_sniffer_service.py ..... [ 74%]
tests\unit\services\test_dirty_waters_service.py ... [ 77%]
tests\unit\services\test_snyk_service.py ..... [ 85%]
tests\unit\services\test_version_classifier.py ..... [ 91%]
tests\unit\test_cli.py ..... [100%]

===== 81 passed in 61.65s (0:01:01) =====

```

Figura 4.7: Execução dos testes

4.3 Avaliação da solução

Esta seção tem como objetivo avaliar requisitos de qualidade da ferramenta global realizada a fim de verificar se o tamanho das dependências de um projeto fazem com que o tempo de execução aumente, bem se os vários ambientes que a ferramenta suporta diferem muito, para quantidades de dependências semelhantes, no desempenho da análise dos *smells*.

Para realizar a avaliação da solução foram utilizados vários projetos exemplo com diferentes complexidades, onde o número de dependências desse projeto é variável de modo a avaliar o desempenho e eficiência da ferramenta.

4.3.1 Tempos de execução da ferramenta

É previsível que com o aumento do número de dependências de um projeto, seja preciso mais tempo para que a detecção dos *smells* seja feita, formando previsivelmente uma gráfico diretamente proporcional entre o número de dependências e o tempo de execução da ferramenta. A tabela seguinte permite comparar os esses mesmos valores entre os diferentes repositórios do GitHub.

Repositório	Número de Dependências	Tempo de execução (s)
RafiLS/TaskReact	739	29.95
axios/axios	714	34.54
ngoworldcommunity/NGOWorld	1891	70.25
Njong392/Abbreve	427	42.95
yubo/react-example	1241	140.44
react/create-react-app	2589	85.98

Tabela 4.1: Tempo de execução para cada Repositório

Para a avaliação da ferramenta global foram usados alguns projetos, um da minha autoria com dependências não usadas e outros *smells* propositalmente para a credibilização do projeto e outros projetos *open source* para observar o comportamento em projetos de diferentes comprimentos. O repositórios utilizados para a avaliação da ferramenta estão apresentados nos tópicos seguintes:

- RafiLS/TaskReact: projeto da minha autoria que permite criar, eliminar e editar uma lista de tarefas.
- axios/axios: é uma biblioteca JavaScript usada para fazer requisições HTTP (*HyperText Transfer Protocol*)
- ngoworldcommunity/NGOWorld: repositório para realizar um ponto de encontro para usuários se conectarem com ONGs (Organização Não Governamental), instituições de caridade e outras organizações por boas causas!
- Njong392/Abbreve: Projeto que contém um dicionário de gírias de código aberto;
- yubo/react-example: Projeto com exemplos simples de aplicação React usado para demonstrar estrutura básica, componentes e configuração;
- react/create-react-app: Ferramenta oficial para criar aplicações React rapidamente com configuração automática de *build*, desenvolvimento e testes.

Para observarmos melhor o impacto que a quantidade de dependências tem no tempo de detecção dos *smells* foi construído um gráfico onde o eixo das abcissa (x) representa o número de dependências de um projeto e o eixo das ordenadas (y) mostra o tempo de execução da ferramenta baseado nos projetos testados e descritos em cima.

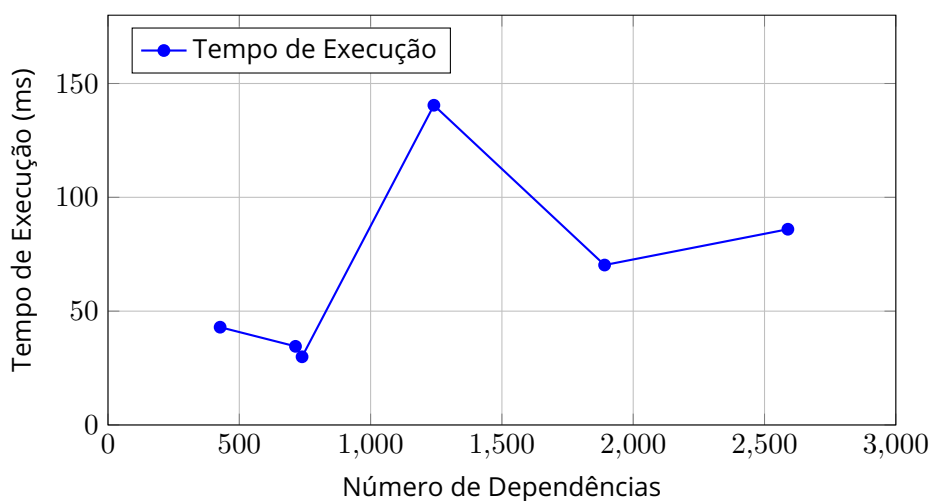


Figura 4.8: Relação entre o número de dependências e o tempo de execução

A reta de regressão linear mostra, que embora para um projeto os resultados sejam um pouco diferentes, os restantes mostram que a ferramenta tende a analisar as dependências seguindo uma proporcionalidade direta, quanto maior for o número de dependências maior é o tempo que a ferramenta demora para executar a sua análise. A análise individual da ferramenta Dirty Waters é a mais demorada pois está associada ao número de pedidos externos realizados a APIs e repositórios remotos durante o processo de análise, enquanto as outras ferramentas funcionam localmente. Uma possível razão para os tempos de execução da ferramenta serem muito elevados quando comparados com a reta de regressão linear poderá vir da estrutura da árvore de dependências e da ferramenta Dirty Waters.

As outras ferramentas também podem ter impacto dependendo da estrutura da árvore de dependências, mas pelos testes feitos a cada ferramenta individualmente, o facto do Depcheck e do Snyk trabalharem localmente, o seu processo de execução é mais rápido.

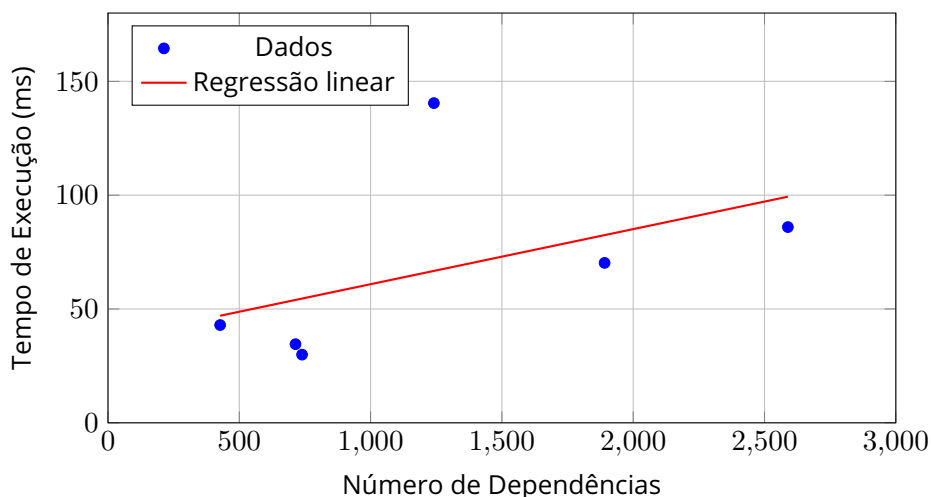


Figura 4.9: Reta de regressão linear entre o número de dependências e o tempo de execução

Assim, podemos concluir que o número de dependências tem impacto no tempo de execução das ferramentas mas não é o único fator a ser levado em conta. O número de

acessos a fontes externas (como repositórios GitHub e APIs) tem um impacto direto no desempenho, uma vez que ferramentas como o Dirty Waters dependem de múltiplas validações por dependência, incluindo verificação de origem, *commit* SHA, proveniência e assinaturas.

4.3.2 *Smells* detetados pela ferramenta

Para avaliar a veracidade da ferramenta foram testadas em conjunto e em separados as várias ferramentas usadas.

O projeto RafiLS/TaskReact utilizado contém dependências que não são usadas de propósito. As figuras 4.10 e 4.11 mostram os resultados das ferramentas usadas em conjunto e em separado:

```
Unused dependencies
* request
* node-uuid
* left-pad
* uuid
Unused devDependencies
* @types/react
* @types/react-dom
* depcheck
* jest
* supertest
```

Figura 4.10:
Output da
ferramenta
Depcheck

```
Unused dependencies/Bloated Dependencies (9)
• request
• node-uuid
• left-pad
• uuid
• @types/react
• @types/react-dom
• depcheck
• jest
• supertest
```

Figura 4.11: Output da
ferramenta global

Foram adicionadas dependências, como, por exemplo o *"jest"* e o *"supertest"* que é uma dependência usada para fazer testes. Na figura 4.8 mostra a execução da ferramenta Depcheck na linha de comandos e a figura 4.9, a execução da ferramenta.

A figura 4.12 mostra o resultado para dependências transitivas usando a ferramenta Snyk e a figura 4.13 o resultado usando a ferramenta global para o mesmo *smell*. Podemos observar em ambos os casos dependências transitivas como o *"follow-redirects"* e o *"form-data"*.

```
Issues with no direct upgrade or patch:
X Improper Removal of Sensitive Information Before Storage or Transfer [High Severity][https://security.snyk.com/vuln/known-cve/CVE-2019-16032] in follow-redirects@1.15.11
  introduced by axios@1.13.5 > follow-redirects@1.15.11
  This issue was fixed in versions: 1.16.0
X Predictable Value Range from Previous Values [Critical Severity][https://security.snyk.com/vuln/known-cve/CVE-2022-36261] in form-data@2.3.3
  introduced by request@2.88.2 > form-data@2.3.3
  This issue was fixed in versions: 2.5.4, 3.0.4, 4.0.4
X Allocation of Resources Without Limits or Throttling [High Severity][https://security.snyk.com/vuln/known-cve/CVE-2022-36261] in form-data@2.3.3
```

Figura 4.12: Output da ferramenta Snyk



Figura 4.13: Output da ferramenta global

Para a ferramenta Dirty Waters as figuras 4.14 e 4.15, do mesmo modo, mostram os resultados da execução delas em conjunto e em separado.

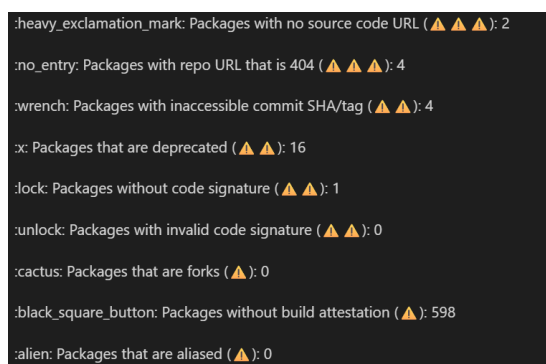


Figura 4.14: Output da ferramenta Dirty Waters

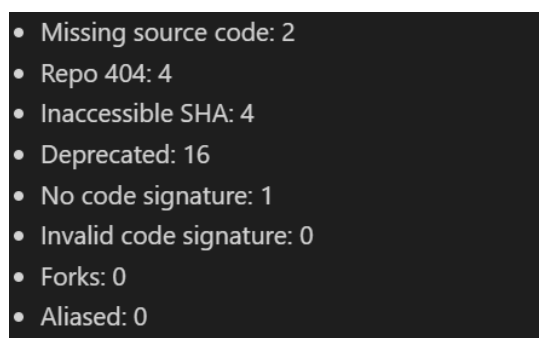


Figura 4.15: Output da ferramenta global

Em cada uma das imagens anteriores é possível comparar os resultados obtidos pela execução das ferramentas externas utilizadas e pelo projeto realizado, que deverão ser similares.

Com base nos testes feitos usando as várias ferramentas usadas, o resultado da ferramenta realizada deve espelhar os resultados das ferramentas individualmente. As ferramentas utilizadas, deste modo, surgem como um género de caixa preta, pois, cada uma das ferramentas foram executadas e o resultado delas é obtido e replicado no relatório de deteção de *smells*, não tendo acesso ao modo de como cada uma das ferramentas executa a sua análise.

4.3.3 Testes da configuração da ativação e desativação da deteção dos *smells*

Nesta secção é demonstrado para o projeto RafiLS/TaskReact, no ficheiro de configuração (config.toml) é possível desativar e ativar a deteção de certos *smells* com *true/false* nos *checks* que o utilizador defenir.

No exemplo testado, é ativado o *smell* chamado "No code signature" e nas figuras 4.16 e 4.17 é possível observar que o *smell* foi detetado enquanto que nas figuras 4.18 e 4.19 esse *smell* foi desativado.

```
check_source_code = true
check_source_code_sha = true
check_deprecated = true
check_forks = true
check_provenance = true
check_code_signature = true
check_aliased_packages = false
```

Figura 4.16: Configurações "Code signature" ativada

```
- Missing source code: 2
- Repo 404: 4
- Inaccessible SHA: 4
- Deprecated: 16
- No code signature: 1
- Invalid code signature: 0
- Forks: 0
- Aliased: 0
```

Figura 4.17: Resultados com a configuração "Code signature" ativada

Para as duas figuras, 4.18 e 4.19, o *smell* de *No Code Signature* é desativado, resultando no valor "0" para esse tópico.

```
check_source_code = true
check_source_code_sha = true
check_deprecated = true
check_forks = true
check_provenance = true
check_code_signature = false
check_aliased_packages = false
```

Figura 4.18: Configurações "Code signature" desativada

```
- Missing source code: 2
- Repo 404: 4
- Inaccessible SHA: 4
- Deprecated: 16
- No code signature: 0
- Invalid code signature: 0
- Forks: 0
- Aliased: 0
```

Figura 4.19: Resultados com a configuração "Code signature" desativada

É importante clarificar que se aparecer o valor "0", não significa que o *smell* está desativado. O valor "0" indica que o projeto não tem risco nesse tópico se e só também, na configuração esse *smell* estiver ativado. Se tiver desativado o valor vai aparecer sempre a zero.

Uma limitação da ferramenta é que a possibilidade de ativar e desativar *smells* que não sejam da ferramenta externa Dirty Waters. Essa possibilidade só está presente para os *smells* dessa ferramenta, por isso se para qualquer ferramenta externa que não seja o Dirty Waters, se o valor aparecer a zero quer dizer que essa ferramenta não encontrou esse tipo de *smell*.

Assim, cabe ao utilizador testar o seu projeto da forma como achar mais conveniente para dependendo do caso melhorar o desempenho da ferramenta alterando os dados no ficheiro de configuração.

Capítulo 5

Conclusões

Este capítulo tem como objetivo apresentar um resumo dos resultados do trabalho desenvolvido, abordando concretizados, limitações, desenvolvimentos futuros e uma apreciação final sobre o trabalho desenvolvido.

5.1 Objetivos concretizados

Os objetivos deste trabalho foram contribuir para a melhoria da segurança e manutenção de projetos com o estudo do conceito de *Software Supply Chain Smell*, clarificação e definição de critérios para detetar *smells*. Foram analisados *smells* associados as dependências e desenvolver um mecanismo automatizado para analisar ficheiros de manifesto, como por exemplo, `package.json`. Foram também concluídos os objetivos relacionados com a procura de ferramentas compatíveis a fim de criar um projeto que junte essas ferramentas e gerar um relatório com a análise dos *smells* pretendidos. O objetivo deste trabalho passa pela análise de várias ferramentas existentes para deteção de *Software smells*, selecionando e integrando aquelas que permitem abordar os *smells* identificados para o problema. Pretende-se desenvolver uma ferramenta mais completa, capaz de identificar diferentes tipos de *smells* relacionados com dependências de software.

Foram feitas também validações, usando testes unitários, de integração e de sistema. Também, a ferramenta foi testada com projetos reais para verificar tanto o tempos de execução e credibilidade da ferramenta.

O trabalho ficou disponível num repositório público da minha autoria, do GitHub, (<https://github.com/RafiLS/DependencyTool.git>), com o intuito de melhorar a segurança e o tempo de manutenção de projetos React.

5.2 Limitações e trabalho futuro

Este trabalho apresenta algumas limitações a nível dos pré-requisitos, pois para que cada ferramenta funcione (Dirty Waters, Depcheck e Snyk), cada uma delas tem de estar instalada individualmente. O projeto foi desenvolvido para que se alguma ferramenta falhar o relatório de análise de *smells* é mostrado na mesma mas os *smells* das ferramenta que não funcionam, não são detetáveis.

É necessário o utilizador tem ter um conjunto de pré-requisitos, Node.js instalado, um gestor de pacotes compatível, como o npm, acesso a ficheiros `package.json` para a ferramenta Depcheck.

A ferramenta Dirty Waters foi desenvolvida em Python, sendo assim, é necessário o Python 3 instalado, bem como o gestor de pacotes *pip*, utilizado para instalar a aplicação e as respectivas dependências. Além disso para utilizar a ferramenta em ambiente Windows é necessário utilizar um ambiente virtual (*venv*) para isolar as dependências do projeto. Depende também do GitHub, pois realiza interações com repositórios alojados, incluindo verificação de versões, tags, commits e meta dados das dependências analisadas. Outro requisito para que a ferramenta funcione bem é a criação de um GitHub API Token, utilizado para autenticar os pedidos realizados à API do GitHub e evitar limitações de acesso.

A ferramenta Snyk utilizada para o seu funcionamento normal, necessita também do Node.js instalado, gestor de dependências correspondente ao projeto analisado, como por exemplo o npm e conta autenticada no Snyk.

No contexto global da aplicação no caso de uma ferramenta falhar não impactará as outras, mas os *smells* que essas ferramentas detetam, não são contabilizados esses smells.

Para trabalhos futuros e manter esta ferramenta reutilizável, seria preciso que esta ferramenta tivesse versões que seguissem e fossem compatíveis com as ferramentas usadas ao longo do tempo. A ferramenta foi desenhada para que a adição de novas ferramentas fosse relativamente simples a fim de cobrir uma maior quantidade de *smells*. Uma possibilidade seria também a possibilidade de explorar outros ambientes para além do ecossistema JavaScript, adicionando suporte para projetos de linguagens como, por exemplo, Python e .NET, permitindo que a ferramenta seja aplicada a um maior número de projetos e contextos de desenvolvimento.

Outra possibilidade de expansão seria a integração da ferramenta em da ferramenta em pipelines de integração contínua, permitindo que a análise de dependências fosse executada automaticamente e detetar *smells* em fases mais iniciais do desenvolvimento. A criação de dashboards com métricas estatísticas também seria uma forma de facilitar o programadores a compreender mais facilmente o impacto de determinadas dependências no projeto.

E por fim, com o tempo algumas ferramentas poderão vir a tornarem-se desatualizadas e, então, seria interessante uma manutenção e verificação contínua das ferramentas utilizadas, a fim de otimizar o desempenho da ferramenta.

5.3 Apreciação final

O trabalho permitiu aprofundar conhecimentos sobre a estrutura dos projetos e as suas dependências, com potenciais implicações na segurança. Demonstrei e apliquei os meus conhecimentos adquiridos ao longo da licenciatura.

Desafiei-me a explorar conceitos novos ligados às dependências de projetos, que muitas vezes não se dá a importância merecida e possibilita a falhas de segurança e manutenção que poderão vir a tornar-se vulnerabilidades.

Capítulo 6

Referências

- Anchore (2019), *SBOM-powered software composition analysis*, <https://anchore.com/>
- Brown, S. (2023). *C4 Model for Software Architecture*. Devovx Poland 2023. <https://static.architectis.je/devovxpl2023-c4-model.pdf>
- chains-project (2025), *Dirty-Waters*, <https://github.com/chains-project/dirty-waters>
- Codacy (2024), *Software supply chain security explained*, <https://blog.codacy.com/software-supply-chain-security>
- DataDog (2024), *dd-dependency-sniffer*, <https://github.com/DataDog/dd-dependency-sniffer>
- depcheck (2024), *depcheck*, <https://www.npmjs.com/package/depcheck>
- DevMedia (n.d.), *Design Patterns: Padrões GoF*, <https://www.devmedia.com.br/design-patterns-padroes-gof/16781>
- Fortinet (2025), *Ataque à cadeia de suprimentos da SolarWinds*, <https://www.fortinet.com/resources/cyberglossary/solarwinds-cyber-attack>
- Fowler, M. (2003), *Catalog of Patterns of Enterprise Application Architecture*, <https://martinfoowler.com/eaCatalog/>
- ISO/IEC (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models (ISO/IEC 25010)*. Available at: <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>
- ISTQB (2026), *International Software Testing Qualifications Board*, <https://istqb.org/>
- JSR (2026), *@cyclonedx/cdxgen*, <https://jsr.io/@cyclonedx/cdxgen>
- Kruchten, P. (1995). *The 4+1 View Model of Architecture*. IEEE Software, **12**(6), 42–50. <https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf>
- Liu, R., Bobadilla, S., Baudry, B. e Monperrus, M. (2022), "Detecting Software Supply Chain Smells", *International Conference on Software Engineering*, Pittsburgh, USA, May
- Liu, R., Bobadilla, S., Baudry, B., & Monperrus, M. (2025), "Dirty-Waters: Detecting software supply chain smells", *International Conference on the Foundations of Software Engineering*, Porto Alegre, Brazil, Jun.
- Node.js (2022). *An introduction to the npm package manager*. Disponível em: <https://nodejs.org/learn/getting-started/an-introduction-to-the-npm-package-manager>

- Noronha, R. (2025), OWASP A03:2025: Why supply chain security is now ranked #3 (and what operators must do), SBOM Insights, <https://sbom-insights.dev/posts/supply-chain-security-voted-a-top-concern-in-2025/>
- Npm (2026), @cyclonedx/cyclonedx-npm, <https://www.npmjs.com/package/@cyclonedx/cyclonedx-npm>
- Nunes, A. (2023), *Teste unitário e o padrão AAA (Arrange, Act, Assert)*, Medium, <https://medium.com/@alamonunes/teste-unit%C3%A1rio-e-o-padr%C3%A3o-aaa-arrange-act-assert-cb81d587368a>
- OpenSSF (2026), *Scorecard - Security health metrics for Open Source*, GitHub, <https://github.com/ossf/scorecard>
- OWASP Foundation (2026), *CycloneDX: Open Standard for Software Bill of Materials (SBOM)*, <https://owasp.org/www-project-cyclonedx/>
- Protocol Buffers Documentation (2023), *Protocol Buffer basics: Java*, <https://protobuf.dev/getting-started/javatutorial/>
- SoftwareSeni (2026), Software supply chain security: From regulatory compliance to incident response, <https://www.softwareseni.com/software-supply-chain-security-from-regulatory-compliance-to-incident-response/>
- Snyk (n.d.), *Plataforma de segurança para desenvolvedores baseada em IA*, <https://snyk.io/pt-PT/>
- webpro-nl (2026), *Knip*, GitHub, <https://github.com/webpro-nl/knip>
- W3Schools (2019), JavaScript JSON, <https://www.w3schools.com/js/>
- W3Schools (2019), What is XML?, <https://www.w3schools.com/xml/>